

쿨러가 더러워진 날

유압 설비의 기름 온도 하나가 31까지 올라갔다가, 사람의 승인을 받아 다시 설비로 내려와 식기까지 — 구조와 데이터의 길

이 문서를 읽는 법

이 문서는 두 가지로 되어 있습니다. 시스템 전체를 빠짐없이 담은 **그림 한 장**, 그리고 그 그림 위를 처음부터 끝까지 지나가는 **이야기 하나**입니다. 숫자보다 **구조**(어떤 부품이 무엇을 하는가), **연결**(누가 누구에게 어떤 프로토콜로, 어느 방향으로 말하는가), **데이터의 길**(어떤 데이터가 어디서 태어나 어떻게 이름과 모양이 바뀌고, 어디에 저장되고, 다음에 누가 읽는가)을 따라갑니다.

이야기 속 동그라미 숫자(예: ⑥)는 그림의 화살표 번호입니다. 숫자를 따라가면 지금 이야기가 그림의 어디를 지나는지 알 수 있고, 이야기를 끝까지 읽으면 그림의 화살표를 하나도 빠짐없이 한 번씩 지나가게 됩니다. 괄호 안에는 그 구간의 프로토콜 · 제품 · 토픽을 적었습니다.

이 설비는 교육용입니다. 진짜 유압 설비 대신 컴퓨터가 열의 흐름을 계산하고, 설비 시간은 실제보다 빨리 흐릅니다. 그 위의 게시판 · 장부 · 검사관 · 기록 DB · 지식 지도 · AI · 업무 진행은 실제로 돌아가는 제품과 프로토콜 그대로입니다.

업무 진행에는 두 방식이 있습니다. 코드에 순서가 박힌 **기본 방식(legacy)**과, BPMN 정의서가 순서를 정하는 **인스턴스 방식(instance)**입니다. 설정의 기본값은 기본 방식이지만, 지금 이 저장소의 설정 파일은 인스턴스 방식으로 켜 둡니다. 앞부분(1~18장)은 기본 방식으로 한 바퀴를 돌고, 19~21장에서 같은 일을 인스턴스 방식으로 다시 봅니다. 두 방식 모두 설비로 내려가는 명령의 길(15장)은 같습니다.

아래층(설비 · 게시판 · 장부 · 검사관 · 기록 DB)은 일부러 가볍게 만들었고, 이 시스템의 무게는 위층에 있습니다. 지식 지도의 일곱 층, 네 가지 판단, BPMN 정의서로 움직이는 업무 진행과 그 버팀 장치, AI 직원과 MCP 서버, 회사 시스템, 사람이 지식을 고치는 길입니다. 그래서 8~11장은 지식 지도를 층마다 · 고장마다 · 판단마다 펼치고, 뒤쪽 장들은 업무 진행 안의 장치를 하나씩 「무엇이 들어와 → 어떻게 바뀌어 → 어디 저장되고 → 누가 읽나」로 따라갑니다.

- | | |
|---------------------------|--|
| 1. 시스템 아키텍처 한 장 | 7. 5장 · 사무실의 장부와 검사관 |
| 2. 서막 · 설비가 깨어날 때 | 8. 6장 · 기록 DB, 그래프, 시스템 감시 |
| 3. 1장 · 유압 설비와 열의 저울 | 9. 7장 · 쿨러가 더러워진다 |
| 4. 2장 · 설비 옆의 PLC | 10. 8장 · 지식 지도 ①: 스키마와 가치 · 프로세스 · 리소스 |
| 5. 3장 · 수집 장치, 엣지, 공장 게시판 | |
| 6. 4장 · 공장과 사무실 사이의 두 문 | |

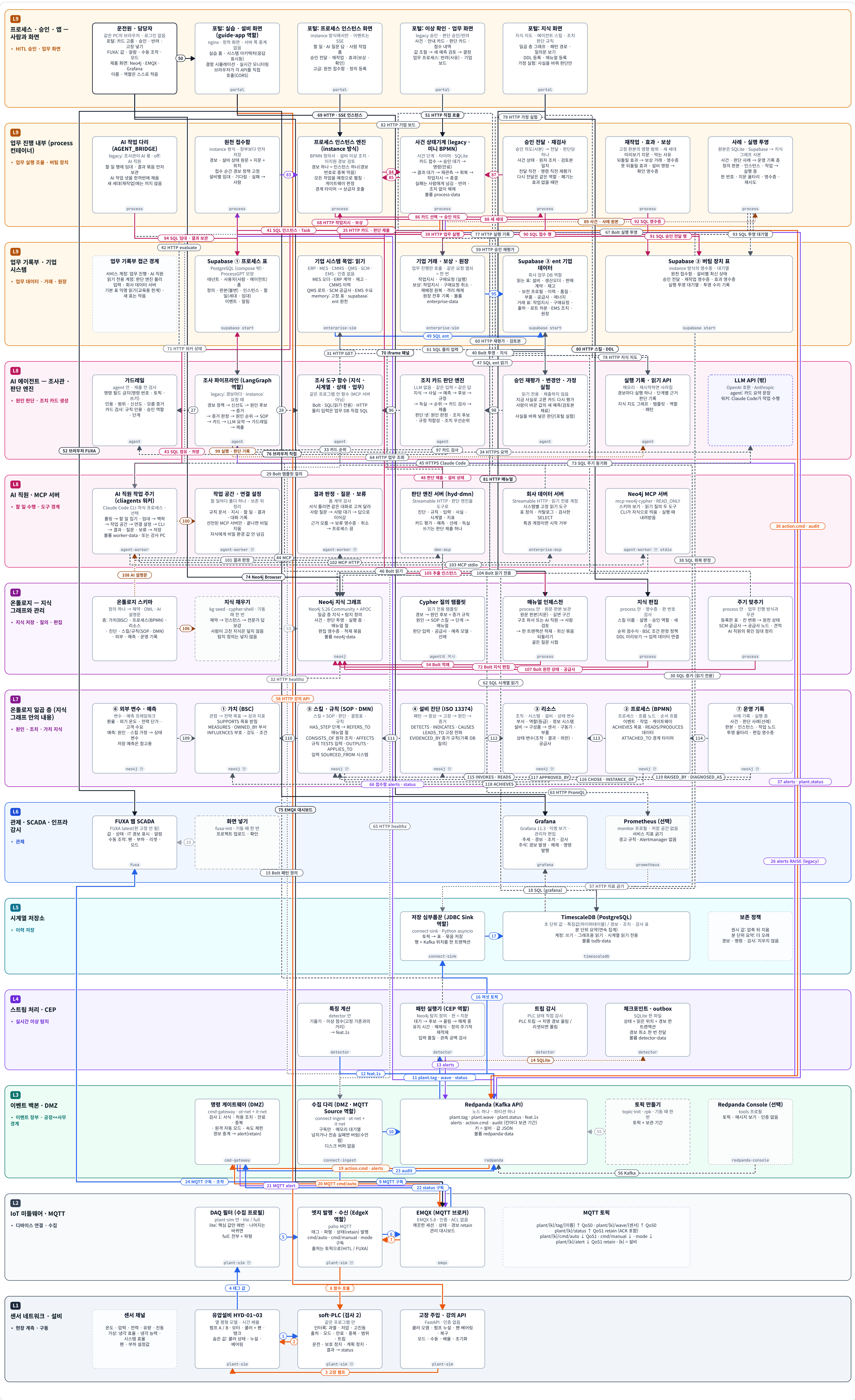
11. 9장 · 지식 지도 ②: 진단 · 스킬과 규칙 · 외부 · 운영 기록
12. 10장 · 지식 지도 ③: 고장마다 사슬 따라가기
13. 11장 · 판단 정의 넷: DMN으로 본 판단
14. 12장 · AI 조사관: 원인 진단
15. 13장 · 판단 엔진: 후보 → 규정 → 순위
16. 14장 · 사람이 결정한다
17. 15장 · 다시 설비로: 검사 두 겹
18. 16장 · 돌아오는 대답, 회복, 종결
19. 17장 · 회사 시스템: 여섯 시스템 · 거래 · 보상 · 원장
20. 18장 · 다른 두 고장과 트립

21. 19장 · BPMN 정의서로 본 같은 흐름: 인스턴스
22. 20장 · 인스턴스가 버티는 법: 접수함 · 승인 전달 · 재작업 · 보상 · 투영
23. 21장 · AI 직원과 MCP 서버
24. 22장 · 포털 화면들
25. 23장 · 지식을 고치는 길: 스킬 · 점수식 · BSC 조건 · DDL · 공급사
26. 24장 · 매뉴얼이 지식이 되는 길
27. 끝 · 순조롭지 않을 때: 갈림길과 실패 경로
28. 요약 · 데이터 리니지 요약
29. 한계 · 알려진 한계
30. 저장소 밖 사실의 출처

시스템 아키텍처 한 장

따 하나가 중 하나입니다(위 L9 업무 · 사람 → 아래 L1 설비). 하는 일이 많은 L9 · L8 · L7은 같은 줄을 떠 여러 개로 나뉘었고, 띠 왼쪽의 줄 번호가 같습니다. 카드는 하는 일 단위이고, 카드 아래 화색 그리드가 실제 컨테이너 · 포트, 카드 마지막 줄이 불립니다. 화살표 119개에는 번호와 프로토콜 · 토크 · 표 이름을 붙였고, 같은 번호가 뒤의 이야기에 나옵니다. 구성요소 69개 · compose 서비스 23개 · 불륨 7개 · 망 2개 · Kafka 토크 7개가 모두 이 그림에 이름으로 들어 있습니다(빌드가 대조).

→ 데이터(상향) → 경보 · 분석 → 조치 명령(하향) → 승인 · 업무 -----> 조회 · 호출 -----> 감시 -----> 기동 · 회 → 사람



서막 설비가 깨어날 때

모든 부품은 Docker Compose 한 파일로 켜집니다. 부품마다 「프로필」이라는 묶음 이름이 붙어 있고, 기본으로 켜는 묶음은 공장(ot) · 장부와 기록(backbone) · 검사(detector) · 지식(knowledge) · AI(agent) · 회사 시스템(enterprise) · 업무 진행(process)입니다. 시스템 감시(monitor), 장부 들여다보기 화면(tools), AI 직원과 MCP 서버(cliagents)는 선택이라 이름을 더해야 켜집니다. 업무 기록부 Supabase는 이 파일 밖에서 Supabase 자체 도구로 따로 켜집니다. 망은 둘입니다. 공장 망(ot-net)과 사무실 망(it-net). 모든 화면과 포트는 이 PC 안에서만 열립니다. 계속 도는 부품은 꺼지면 스스로 다시 켜지고, 각자 메모리 상한이 있습니다.

켜질 때 잠깐 일하고 꺼지는 도우미 셋이 먼저 움직입니다. 첫째는 사무실 장부 Redpanda에 칸(토픽)들을 보관 기간과 함께 만듭니다 (55). 둘째는 지식 지도 Neo4j에 제약 조건을 걸고 온톨로지 인스턴스(설비 · 센서 · 이상 패턴 · 증상 · 고장 · 원인 · 증거 규칙 · 조치 방법(SOP) · 판단 규칙 · 회사 지표)를 채운 뒤, 전문가 질문에 대한 답(유량과 생산량의 관계, 작동유 열화, 저압 · 고진동 트립)으로 보강합니다(Bolt) (53). 여러 번 돌려도 결과가 같고, 사람이 화면에서 고친 지식은 덮어쓰지 않으며, 옛 판 지식 지도를 발견하면 한 번 지우고 새 판으로 채웁니다. 단, 이 도우미는 검사관이 쓰는 「탐지 정의」(어떤 조합을 얼마나 유지하면 경보로 볼지, 언제 풀지)를 넣지 않습니다(알려진 한계). 셋째는 운전실 화면 FUXA에 태그 · 화면 · 알람을 담은 프로젝트를 올리고 확인합니다(HTTP) (25). FUXA는 저장 공간이 없어 켜질 때마다 이 도우미가 다시 넣습니다.

표준 제품 자리와 hyd에서 실제로 도는 것. 이 시스템의 그림은 실제 공장 IoT · SCADA 설계의 자리(층)를 그대로 따르지만, 무거운 제품은 같은 이름 · 같은 메시지 모양을 지키는 작은 Python 서비스로 바꿨습니다. 그래서 이야기 속 「~역할」이라는 말은 「그 제품이 앉을 자리에 이것이 대신 앉아 있다」는 뜻입니다. 아래층부터 보면 이렇습니다. 실제 설비 · PLC · 수집 장치 · EdgeX 같은 엣지 플랫폼 자리는 설비 프로그램(plant-sim) 하나가 함수 호출로 대신하고, 밖으로는 진짜 MQTT 토픽과 쪽지 모양을 지킵니다. MQTT 브로커 자리는 진짜 제품 EMQX, 운전실 화면은 진짜 FUXA입니다. 공장 게시판에서 장부로 옮기는 Kafka Connect 소스 커넥터 자리는 수집 다리(connect-ingest), 장부에서 기록 DB로 옮기는 JDBC 싱크 커넥터 자리는 저장 심부름꾼(connect-sink)이고 둘 다 Python입니다. Kafka 자리는 Kafka API를 그대로 쓰는 진짜 제품 Redpanda입니다. Flink의 패턴 인식(CEP) 작업과 짧은 창 특징 계산 작업, 학습한 오토인코더(ONNX) 이상 점수 자리는 검사관(detector) 하나가 대신합니다. 패턴 인식은 명시적인 상태 기계로, 이상 점수는 학습 모델이 아니라 고정된 정상 기준과의 거리로 계산합니다. 이 시스템 어디에도 Flink는 없습니다. 기록 DB는 InfluxDB가 아니라 PostgreSQL 기반의 진짜 TimescaleDB이고, Grafana와 Prometheus도 진짜 제품입니다. 다만 경고를 사람에게 보내는 Alertmanager는 없습니다. 지식 지도는 진짜 Neo4j입니다.

위층으로 가면 섞여 있습니다. LangGraph로 짜는 AI 에이전트 자리는 조사관(agent) 한 프로그램이 같은 순서(조사 단계 → 가드레일 → 제출)를 지키는 보통 Python 코드로 대신하고, LangGraph 라이브러리는 쓰지 않습니다. 조사관이 쓰는 「도구」도 별도 MCP 서버가 아니라 같은 프로그램 안의 함수입니다. 반면 AI 직원 쪽의 MCP 서버는 지금 진짜로 따로 도는 서버들입니다. 판단 엔진 서버(dmn-mcp)와 회사 데이터 서버(enterprise-mcp)는 각자 컨테이너로 Streamable HTTP를 내놓고, Neo4j MCP 서버는 공개 제품(mcp-neo4j-cypher)을 그대로 씁니다. 저장소 안내문에 아직 「MCP 서버 분리 범위 밖」이라고 적힌 줄은 지금 코드보다 옛날 이야기입니다. 여러 언어 모델을 하나로 묶는 중계 LiteLLM 자리는 조사관이 OpenAI 호환과 Anthropic 가운데 직접 고르는 설정이 대신하고, 원하면 OpenAI 호환 주소를 강사가 띄운

LiteLLM이나 SGLang으로 돌릴 수 있습니다. ProcessGPT 자리는 업무 진행(process)의 인스턴스 엔진이 맡습니다. ProcessGPT 제품을 띄우지는 않지만 업무 기록부 Supabase의 표 모양 · 상태 이름 · 할 일을 집고 결과를 내는 절차는 제품과 같게 만들었고, AI 직원은 ProcessGPT를 만든 쪽의 공개 라이브러리(cliagents)로 Claude Code를 돌립니다. 정리하면 hyd는 층과 연결(이름 · 토픽 · 쪽지 모양 · 표 모양)은 실제 설계대로 두고 아래층 구현을 가볍게 바꾼 시스템이고, 형제 프로젝트 ceco는 같은 자리에 실제 제품을 더 많이 띄웁니다. 대신 hyd는 그 가벼운 아래층 위에 온톨로지 · 판단 · 업무 진행 · AI 직원 층을 훨씬 크게 쌓았습니다.

지금 설정 파일이 고른 것은 이렇습니다. 수집은 가벼운 보고(lite), 업무 진행은 인스턴스 방식, AI 똬은 조사관이 대신하는 다리(legacy 다리), 회사 시스템의 데이터는 업무 기록부 Supabase, 요약 문장을 쓰는 언어 모델은 OpenAI 호환 주소. 똬은 해당 장에서 나옵니다.

1장 유압 설비와 열의 저울

유압 설비는 기름을 높은 압력으로 밀어 기계를 움직이는 심장입니다. 모터가 펌프를 돌리고, 펌프가 탱크의 기름을 밀어내고, 일을 마치고 데워진 기름은 쿨러에서 팬 바람으로 식은 뒤 탱크로 돌아옵니다. 펌프는 주 펌프 A와 예비 펌프 B가 있고, 같은 구조의 설비가 HYD-01 · 02 · 03으로 여러 대 있습니다.

핵심은 **열의 저울**입니다. 펌프가 세게 일할수록(부하) 열이 나고, 쿨러가 팬 속도와 쿨러 상태에 따라 열을 뺍니다. 두 힘이 맞는 곳에서 기름 온도가 멈춥니다. 쿨러 입구 기름 온도 **TS1**이 이 이야기의 주인공 값입니다. 쿨러 핀에 먼지가 끼면 식히는 힘이 떨어져 TS1이 오릅니다.

열 모델에는 천천히 나빠지는 **숨은 값**이 셋 있고, 각각 고장 하나에 대응합니다. 쿨러 상태(먼지 오염), 주 펌프 A의 내부 누설(축 씰 마모, 예비 펌프 B는 새지 않음), 팬 베어링 마모. 셋 다 지식 지도의 「외란 상태 변수」와 같은 개념입니다. 센서 값은 모두 이 숨은 값과 부하 · 팬 · 기름 온도에서 계산됩니다. 온도(쿨러 입구 · 출구 · 탱크 · 캐비닛), 압력(펌프 토출과 보조 압력), 모터 전력, 유량, 진동, 그리고 실제 계기가 아니라 다른 값에서 계산한 가상 센서(냉각 효율 CE · 냉각 능력 CP · 시스템 효율 SE)와 팬 · 부하 설정값입니다. 누설은 압력과 유량을 떨어뜨리고, 베어링 마모는 팬이 빠를수록 진동을 키우고, 쿨러 오염은 냉각 효율을 떨어뜨립니다. 캐비닛 온도는 바깥 온도에 고정된 덧셈이라 사실상 변하지 않습니다(12장에서 다시 나옵니다).

설비 쪽 프로그램(plant-sim, Python) 하나 안에 열 모델 · PLC · 수집 장치 · 엣지가 모두 들어 있고, 서로 네트워크가 아니라 함수 호출로 이어집니다. 실제 현장이라면 설비와 PLC 사이가 필드버스나 Modbus 같은 선이지만, 이 교육용 설비에는 그 구간이 없습니다. 열 모델이 설비 시간을 잘게 쪼개 한 걸음 계산할 때마다 PLC가 값을 봅니다 ①. PLC가 받아들이는 팬 속도 · 부하 · 펌프 선택 · 정지 · 리셋은 곧바로 설비 상태에 반영됩니다 ②.

시간 배율은 켄 때 정해지고, 강의용 버튼으로 바꿀 수 있습니다. 그런데 바뀌도 설비만 빨라지거나 느려질 뿐, 검사관과 업무 진행은 켄 때의 배율로 시간을 계속 셉니다. 배율을 바꾸면 둘이 어긋납니다(알려진 한계).

2장 설비 옆의 PLC

PLC(soft-PLC)는 설비의 마지막 안전장치입니다. 기름이 너무 뜨겁거나, 펌프 압력이 너무 낮거나, 진동이 너무 크면 누가 뭐라 하든 펌프를 세웁니다(트립). 원인은 과열 · 저압 · 고진동 가운데 처음 걸린 하나로 기록됩니다. 트립은 저절로 풀리지 않고, 기름이 충분히 식은 뒤 리셋 명령을 받아야 다시 돕니다. 네트워크가 끊겨도 이 판단은 설비 옆에서 혼자 합니다. 실제 큰 공정에서는 이런 안전 차단을 일반 제어와 독립된 안전 계장 시스템(SIS)으로 따로 두라고 국제 표준 IEC 61511이 요구합니다. 이 교육용 설비는 소프트웨어 PLC 하나가 제어와 인터록을 함께 맡습니다.

PLC의 운전 상태는 운전(RUN), 보호 정지(TRIP), 정지 명령으로 세운 계획 정지(STOP)입니다. 쓸 수 있는 값은 팬 · 부하 · 운전 펌프(A/B) · 정지 · 리셋뿐이고, 압력 설정값은 이 PLC에 없습니다.

PLC는 누가 명령을 보냈는지 따집니다. 운전 모드는 현장(LOCAL), 운전실 수동(REMOTE_MANUAL), 승인된 자동 조치도 받음(REMOTE_AUTO)입니다. 명령이 게시판의 어느 칸으로 왔는지로 보낸 쪽을 정합니다. 자동 명령 칸이면 「사람이 승인한 자동 조치(HITL)」, 수동 명령 칸이면 「운전원」 이고, 쪽지 안에 적힌 출처는 믿지 않습니다. 자동 조치는 원격 자동 모드에서만, 운전원 명령은 현장 모드가 아닐 때만 받습니다. 운전원이 원격 자동 모드에서 수동으로 값을 바꾸면 PLC가 스스로 원격 수동으로 바꿉니다. 수동이 언제나 이기고, 자동 조치를 다시 받으려면 사람이 모드를 되돌려야 합니다. 모드를 바꿀 수 있는 곳은 운전실 화면과 현장 패널(강의용 버튼)뿐입니다.

명령을 받으면 PLC는 차례로 봅니다. 보낸 쪽과 모드가 맞는가 → PLC 시계로 만료되지 않았나 → 최근 명령과 번호가 겹치지 않나 → 아는 값이고 범위 안인가 → 트립 중이면 리셋만, 리셋은 충분히 식었을 때만. 하나라도 걸리면 거절하고 이유를 남깁니다. 결과(완료 또는 거절과 이유)는 따로 보내지 않고 다음 「설비 상태」에 실립니다.

데이터: 설비 상태. PLC가 매초, 그리고 명령을 처리한 직후 만드는 묶음입니다. 운전 모드 · 운전 상태 · 트립 원인 · 기름과 바깥 온도 · 팬과 부하 설정 · 운전 펌프 · 숨은 값 셋 · 진행 중인 고장 · 예측 모델 이름 · 마지막 명령의 번호와 결과와 이유와 출처, 그리고 운전실 화면용 한글 표시가 들어 있습니다. 이 묶음이 나중에 게이트웨이의 모드 확인, 업무 진행의 결과 확인과 예측 입력, 검사관의 트립 감시, 기록 DB의 명령 결과로 쓰입니다.

3장 수집 장치, 엷지, 공장 게시판

엷지의 일꾼이 주기적으로 센서 값 전체를 수집 장치(DAQ 필터)에게 넘기고 이번에 보낼 것을 받습니다 ④⑤. 가벼운 보고(lite)에서는 판단에 꼭 필요한 값(TS1 · CE · PS1 · FS1 · VS1 · 부하 설정)은 매번 보내고, 나머지 핵심 값은 일정 폭 이상 바뀌었거나 일정 시간이 지났을 때만, 보조 채널은 더 드물게 보냅니다. 실제 현장의 기록 장치들이 대역을 아끼는 「바뀌었을 때만 보고」 방식입니다. 전부 보고(full)로 바꾸면 모든 값을 매번 보내고, 압력 · 전력 · 유량의 빠른 파형도 묶음으로 보냅니다.

데이터: 센서 값. 엷지는 값을 공장 게시판 EMQX에 쪽지로 붙입니다(MQTT) ⑥. 칸 이름은 「plant/설비/tag/이름」(예: plant/hyd01/tag/TS1)이고, 쪽지는 {젠 시각, 값, 품질}입니다. 젠 시각은 설비 시간이 아니라 보낸 순간의 실제 시각입니다. 파형은 「plant/설비/wave/센서」 칸에 {시작 시각, 주파수, 값 목록}으로, 설비 상태는 「plant/설비/status」 칸에 붙입니다.

MQTT 게시판은 생중계 방송국입니다. 쪽지가 붙으면 그 칸을 구독해 둔 쪽에게 즉시 밀어 보내고, 보관하지는 않습니다. 계측값과 파형은 QoS 0(한 번 보내고 끝)입니다. 곧 새 값이 또 오니까 한 장쯤 빠져도 된다는 선택입니다. 연결은 모두 「깨끗한 세션」이라 받는 쪽이 끊겨 있던 동안 온 값은 모아 두지 않습니다. 반대로 설비 상태와 경보는 QoS 1(확인이 올 때까지 다시 보냄)에 「칸마다 마지막 한 장 남기기(retain)」로 붙여, 나중에 연결한 쪽도 지금 상태를 바로 압니다. 명령 칸은 남기지 않습니다. 이 교육용 게시판에는 계정 · 칸별 권한이 없습니다. 게시판의 관리 화면(EMQX 대시보드)을 브라우저로 열면 누가 붙어 있고 어느 칸에 쪽지가 오가는지 볼 수 있습니다 (75).

같은 게시판을 운전실 화면(FUXA, 웹 SCADA)이 구독해서 값 · 설비 상태 · IT 경보가 운전원 모니터에 뜹니다 (24). 화면은 「설비 현황」 한 장에 설비별로 경보 표시등, 모드와 PLC 상태, TS1 막대와 주요 값, 수동 조작 상자(팬 · 부하 · 보호 정지 해제 · 모드 선택, 마지막 명령 결과와 거절 사유)가 있고, 유온 알람과 IT 경보 알람이 걸려 있습니다. 운전원은 같은 PC의 브라우저로 로그인 없이 엽니다 (52).

데이터: 운전원 명령. 운전원이 팬 · 부하 · 리셋을 바꾸면 FUXA는 그 값 하나만 담은 쪽지를 게시판의 수동 명령 칸에, 모드를 고르면 모드 번호를 모드 칸에 붙입니다 (24). 엿지가 받아서 (7) 수동 쪽지에 운전원 명령 번호를 새로 붙여 PLC 검사로 넘기고 (8), 결과는 다음 설비 상태에 실려 다시 화면에 돌아옵니다. 여기까지는 모두 공장 망 안입니다. 사무실 쪽이 통째로 멈춰도 화면 · 수동 조작 · 트립은 그대로 둡니다.

4장 공장과 사무실 사이의 두 문

이제 값이 공장 망에서 사무실 망으로 넘어가야 합니다. 두 망이 아무 데나 연결되면 사무실이 뚫렸을 때 공장까지 뚫립니다. 그래서 두 망에 함께 붙는 서비스를 딱 둘로 정했습니다. 산업 보안 표준 IEC 62443은 이런 경계를 전용 구역(DMZ)과 관문(conduit)으로 만들라고 하지만, 이 교육용 시스템은 가볍게 돌리려고 줄였습니다. 방화벽 장치나 전용 망은 없고, 두 망에 동시에 붙은 서비스 둘이 그 역할을 하며, 방향은 장치가 아니라 프로그램 규칙으로 지킵니다.

올라가는 문, 수집 다리(connect-ingest)는 공장 게시판을 구독만 합니다(MQTT) (9). 쪽지를 받으면 칸 이름에서 설비(HYD-01)와 값 이름(TS1)을 꺼내 쪽지 안에 넣고, 사무실 장부 Redpanda의 칸에 설비 이름(hyd01)을 열쇠로 적습니다(Kafka 프로토콜) (10). 이름이 이렇게 바뀝니다. 「plant/.../tag/...」 → plant.tag, 「.../status」 → plant.status(원문 그대로 + 설비), 「.../wave/...」 → plant.wave. 공장 쪽에는 아무것도 쓰지 않습니다. 받는 일꾼과 적는 일꾼 사이에는 메모리 대기열만 있어, 장부가 느려 가득 차거나 적기에 실패한 쪽지는 버리고 버린 수만 셉니다. 디스크 버퍼는 없습니다. 수집 다리는 「마지막 쪽지를 받은 뒤 얼마나 지났나」를 상태로 내놓아, 조사관의 신선도 판단과 시스템 감시가 이것을 봅니다.

내려가는 문, 명령 게이트웨이(cmd-gateway)는 사무실에서 공장으로 내려가는 유일한 네트워크 문입니다. 15장에서 자세히 나옵니다.

다만 두 문을 거치지 않는 길이 하나 있습니다. **강의용 입구**입니다. 포털의 「결함 시뮬레이션」 화면은 브라우저에서 설비 프로그램의 강의용 입구로 바로 요청을 보냅니다(HTTP, 인증 없음) (58). 이 입구로 고장을 넣고(쿨러 오염 · 펌프 누설 · 팬 베어링 마모 · 복구), 현장 패널처럼 모드를 바꾸고 값을 쓰고(운전원 명령과 같은 규칙), 모든 설비를 정상으로 되돌리고, 시간 배율을 바꿉니다. 교육용 도구라 명령 게이트웨이의 검사를 거치지 않습니다.

5장 사무실의 장부와 검사관

Redpanda는 Kafka와 같은 방식(Kafka API)으로 쓰는 더 가벼운 장부입니다. 여기서는 노드 하나의 개발용 설정이고 칸마다 파티션 · 복제본이 하나입니다. 들어온 순서대로 번호를 붙여 디스크에 적고, 누가 읽어도 지우지 않습니다. 칸은 일곱입니다. 계측(plant.tag) · 파형(plant.wave) · 설비 상태(plant.status) · 특징값(feet.1s) · 경보(alerts) · 명령(action.cmd) · 감사(audit). 칸마다 보관 기간이 다르고, 계측처럼 기록 DB로 옮기는 것은 짧게, 경보 · 명령 · 감사는 길게 둡니다. 읽는 쪽은 저마다 「어디까지 읽었나」 책갈피를 가지고 있어 서로 모른 채 각자 읽습니다. 설비 이름을 열쇠로 적었기 때문에 한 설비의 값은 순서대로 읽힙니다. 「온도가 계속 오르고 있나」 같은 판단은 순서가 섞이면 틀립니다. 그러니까 「MQTT 대신 Kafka」가 아니라 「MQTT 다음에 Kafka」입니다. 다만 장부에 들어가기 전의 이음새(QoS 0, 깨끗한 세션, 메모리 대기열)는 장부가 지켜 주지 못합니다.

장부 안을 들여다보는 화면 Redpanda Console은 선택입니다. 켜면 칸과 쪽지를 브라우저로 보고, 손으로 쪽지를 넣을 수도 있습니다(로그인 없음) (56).

검사관(detector)은 Flink 같은 스트림 처리 · CEP 역할입니다. 장부에서 계측 · 파형 · 설비 상태를 가장 최근 것부터 읽습니다 (11). **데이터: 특징값**. TS1이 들어올 때마다 최근 구간의 기울기(오르는 중인가)와, 정상 운전점에서 TS1 · CE · VS1이 얼마나 벗어났는지를 합친 이상 점수를 계산해 장부의 feat.1s 칸에 적습니다 (12). 이상 점수는 학습한 모델이 아니라 고정된 정상 기준과의 거리이고(오토인코더 자리를 대신함), 재료 값 중 하나라도 없거나 묵었으면 비워 둡니다. 전부 보고 설정에서는 파형 묶음마다 평균 · 실효값 · 기울기도 같은 칸에 적습니다. 특징값은 장부의 확인을 기다리지 않고 보냅니다.

「무엇을 위험한 조합으로 볼지」는 코드에 박혀 있지 않습니다. 검사관은 지식 지도에 적힌 **탐지 정의**를 켤 때와 주기적으로 다시 읽습니다(Bolt) (15). 정의 하나는 「검사 조건 목록(모두 참이어야 함) · 해제식 · 유지 시간 · 해제 유지 시간 · 기울기 창」입니다. 검사관은 정의를 실행 코드로 바꾸지 않고, 비교 · 그리고/또는/아님 · 기울기만 허락된 작은 문법으로 읽습니다. 매번 보고되지 않는 값을 쓰는 정의나 형식이 틀린 정의가 하나라도 있으면 묶음 전체를 받지 않고 쓰던 것을 계속 씁니다. 정의마다 내용으로 만든 지문이 판 번호가 되어, 정의가 바뀌면 유지 시간을 채우던 후보는 처음부터 다시 세고, 이미 올린 경보는 풀릴 때까지 원래 정의로 봅니다. 지식 지도를 읽지 못하면 「준비 안 됨」을 알리고, 이미 올린 경보만 계속 지켜볼 뿐 새 후보는 세지 않습니다.

패턴마다 「대기 → 후보 → 올림 → 해제 중」 네 단계를 지냅니다. 조건이 유지 시간 동안 이어지면 올리고, 해제식이 유지 시간 동안 이어지면 풀립니다. 올리는 선과 푸는 선을 다르게 두어(히스테리시스) 값이 경계에서 흔들려도 경보가 깜빡이지 않습니다. 유지 시간은 설비 시간으로 셉니다. 검사관은 들어온 값을 그냥 믿지 않습니다. 시각이 이상하거나(읽을 수 없음 · 미래 · 같은 시각에 다른 값), 품질이 나쁘거나, 숫자가 아니면 무효이고, 보고 주기보다 오래 끊기면 「관측 공백」입니다. 무효나 공백이 생기면 채우던 유지 시간을 처음으로 돌립니다. 모르는 동안을 「계속 참이었다」고 치지 않기 위해서입니다. 올린 경보는 모르는 값만으로 풀리지 않습니다.

검사관은 자기 상태 · 읽은 위치 · 보낼 경보를 한 파일(SQLite)에 한 번에 저장하고 (14), 경보는 장부가 받았다는 확인을 받은 뒤에야 지웁니다(outbox). 그래서 꺼졌다 켜져도 경보를 읽지 않고, 같은 경보가 한 번 더 갈 수는 있습니다. 다시 켤 때는 꺼져 있던 동안을 유지 시간에 넣지 않고, 저장된 시간 배율이나 수집 설정이 지금과 다르다면 시작하지 않습니다.

6장 기록 DB, 그래프, 시스템 감시

저장 심부름꾼(connect-sink, JDBC Sink 역할)은 장부의 여섯 칸(계측 · 특징값 · 경보 · 명령 · 설비 상태 · 감사)을 처음부터 읽어 (16) 기록 DB TimescaleDB(PostgreSQL 기반)로 옮깁니다(SQL) (17). 묶음으로 받아, 표에 쓰는 일과 「어디까지 옮겼나」 기록을 한 트랜잭션으로 저장한 뒤에야 장부에 「다 읽었다」 고 알립니다. DB가 잠깐 죽었다 살아나도 빠뜨리거나 두 번 적지 않습니다. 형식이 틀린 쪽지는 그것만 건너뛰고 씩니다.

옮기면서 데이터의 모양이 바뀝니다. 계측 → 초 단위 값 표(품질 칸은 버림), 특징값 → 특징값 표, 경보 → 경보 표(올림 때 한 줄을 만들고 풀림 때 같은 줄을 고침), 명령 → 조치 표(한 줄), 설비 상태 → 그 안에 실린 PLC 결과만 골라 같은 명령 줄에 「처음 받은 결과」로 덧붙임, 감사 → 감사 표. 초 단위 값에서 분 단위 요약(평균 · 최소 · 최대)이 자동으로 만들어집니다. 초 단위 값과 특징값은 시간이 지나면 압축 뒤 지우고, 분 단위 요약은 더 오래, 경보 · 명령 · 감사는 지우지 않습니다. 계정은 셋입니다. 쓰는 계정, 모든 표를 읽기만 하는 그래프용 계정, 시계열 표만 읽는 읽기 전용 계정(AI 쪽이 씀).

Grafana가 이 기록으로 대시보드를 그립니다(SQL) (18). 현재값 · 추세 · 이상 점수 · 활성 경보 · 경보 이력 · 조치 이력(명령 · 승인자 · PLC 결과) · 감사 기록이 있고, 그래프 위에는 경보가 난 때 · 풀린 때 · 명령을 낸 때가 표시됩니다(작업지시만 고른 승인은 명령이 없어 표시되지 않음). 로그인 없이 보기 전용으로 열리고 브라우저로 직접 열 수도 있습니다 (76). 관리자 계정으로 로그인하면 대시보드를 고칠 수 있고, 포털 안에 끼워 넣어 보여 줄 수 있게 허용되어 있습니다.

시스템 감시 Prometheus는 선택입니다. 커면 수집 다리 · 게이트웨이 · 검사관 · 저장 심부름꾼 · 조사관 · 업무 진행이 내놓는 지표(받은 쪽지 수, 대기열, 마지막 쪽지 이후 시간, 게이트웨이 통과 · 거부, 경보 수, 이상 점수 등)를 주기적으로 긁습니다(HTTP) (57). 경고 규칙은 「수집 멈춤」과 「게이트웨이 거부 많음」 둘입니다. 경고를 사람에게 보내는 장치(Alertmanager)는 없어 Prometheus 화면에서만 보이고, 저장 공간이 없어 재시작하면 기록이 사라집니다. 설비 프로그램과 회사 시스템도 지표를 내놓지만 긁지 않습니다.

7장 쿨러가 더러워진다

HYD-01의 쿨러 핀에 먼지가 끼기 시작합니다. 강요용 버튼으로 「쿨러 오염」 고장을 넣으면, 고장 램프가 숨은 값(쿨러 상태)을 천천히 나쁘게 만듭니다 (3). 냉각 효율 CE가 떨어지고, 아무것도 하지 않으면 기름은 계속 올라 결국 PLC가 펌프를 세웁니다.

데이터: 경보. 검사관이 「TS1이 높음 + 냉각 효율 낮음 + TS1 오르는 중」이 유지 시간 동안 이어진 것을 보고 쿨러 열화 경보(올림)를 장부의 alerts 칸에 적습니다 (13). 경보에는 경보 번호 · 설비 · 패턴 · 심각도 · 상태(올림/풀림) · 시각, 그리고 근거(그때의 관련 값과 기율기 · 이상 점수, 적용한 정의와 판 번호, 조건 판정, 값마다 얼마나 묵었는지)가 실립니다. 풀릴 때는 같은 경보 번호로 「풀림」이 나갑니다.

PLC가 트립을 걸면 검사관은 유지 시간을 기다리지 않고 트립 경보(과열 · 저압 · 고진동 정지, 원인을 모르면 PLC 정지)를 따로 냅니다. 이것은 지식 지도의 탐지 정의가 아니라 설비 상태를 직접 지켜보는 별도 감시가 내고, PLC가 리셋되면 같은 번호로 풀림을 냅니다.

경보는 여러 곳으로 갑니다. 첫째, 내려가는 문(게이트웨이)이 경보를 읽어 (19) 운전실 표시용 모양(표시등 단계와 「경보 발생 · 쿨러 성능 저하」 같은 한글 문구)으로 바꿔 게시판의 설비별 경보 칸에 「마지막 한 장 남기기」로 붙입니다(MQTT) (21). 운전실 화면의 IT 경보 표시등이 켜집니다. 그런데 운전실 화면의 IT 경보 알람 문구는 어떤 패턴이든 늘 「쿨러 성능 저하 (Flink CEP RAISE)」로 고정되어 있습니다(이 시스템에 Flink는 없습니다). 정확한 패턴 이름은 화면의 경보 문구 칸에 나옵니다. 둘째, 저장 심부름꾼이 경보 표에 적어 Grafana에 표시가 생깁니다. 셋째, 기본 방식에서는 AI 조사관이 경보로 깨어납니다 (26). 넷째, 인스턴스 방식에서는 업무 진행의 접수함이 경보를 받습니다 (66)(19장).

8장 지식 지도 ①: 스키마와 가치 · 프로세스 · 리소스

AI가 무엇을 근거로 판단하는지 보려면 먼저 지식 지도(Neo4j, APOC 확장)를 알아야 합니다. 지식 지도는 표가 아니라 「개념과 개념 사이의 관계」를 선으로 이은 그래프이고, 그 뼈대가 온톨로지 스키마입니다. 스키마 정의는 층 · 클래스 · 관계 · 속성을 적은 문서 하나이고, 여기서 세 가지가 만들어집니다. Neo4j에 거는 제약 조건(같은 아이디가 두 번 생기지 않게), 표준 온톨로지 언어로 쓴 OWL 문서, 그리고 AI 직원에게 주는 설명문입니다. 제약 조건은 지식 채우기 도우미가 가장 먼저 겁니다 (106). 설명문은 AI 직원의 이미지와 지시에 함께 들어가 「이 그래프에는 어떤 노드와 관계가 있다」를 알려 줍니다 (108). 실제 인스턴스는 사람이 쓴 파일로 채웁니다. 층은 일곱이고, 층마다 국제 표준이나 경영 기법을 본뒀습니다. 지식 지도에 든 회사는 가상 기업 「경남유압」이고, 규정과 매뉴얼은 교육용 예시입니다. 브라우저로 Neo4j Browser를 열면 이 그래프를 직접 볼 수 있습니다 (74). 그림에서는 일곱 층이 「온톨로지 일곱 층」 띠의 카드 일곱 장이고, 카드 사이 화살표에는 층과 층을 잇는 관계 이름이 적혀 있습니다.

① **가치 층(BSC, 균형성과표)**. 회사가 무엇을 얻고 잃는지입니다. 맨 위 뼈대는 관점이고, 학습과 성장 → 내부 프로세스 → 고객 → 재무 순서로 아래에서 위로 쌓입니다. 관점마다 전략 목표가 속합니다(IN_PERSPECTIVE). 재무 관점에는 「영업이익 확대」와 「원가 절감」, 고객 관점에는 「납기 신뢰」와 「품질 신뢰」, 내부 프로세스 관점에는 「설비 가용성 확보」·「무재해 운전」·「에너지 효율」, 학습과 성장 관점에는 「현장 판단 지식 축적」이 있습니다. 목표끼리는 받침 관계(SUPPORTS)로 이어집니다. 현장 판단 지식 축적과 무재해 운전이 설비 가용성을 받치고, 설비 가용성이 납기 신뢰를, 에너지 효율이 원가 절감을 받치며, 납기 신뢰 · 품질 신뢰 · 원가 절감이 함께 영업이익 확대를 받칩니다. 「무재해 운전」에는 「인터록 한계를 넘지 않는다, 금액과 맞바꾸지 않는다」는 설명이 붙어 있습니다.

성과 지표는 저마다 한 목표를 재고(MEASURES) 담당 부서가 있습니다(OWNED_BY). 영업이익 확대는 영업이익(공장 경영)과 매출(영업팀)로 켜집니다. 원가 절감은 총비용(공장 경영) · 보전비(설비보전팀) · 에너지비(환경안전 · 에너지팀) · 부품 구매비와 부품 단가(구매팀) · 재고 보관비와 재고량(생산팀) · 지체상금(영업팀)으로 켜집니다. 납기 신뢰는 납기 준수율(영업팀), 품질 신뢰는 품질 클레임(품질팀) · 브랜드 신뢰(영업팀) · 부품 품질(구매팀), 설비 가용성은 설비 가동률 · 생산량(생산팀)과 평균 고장 간격 · 작동유 잔여 수명(설비보전팀), 무재해 운전은 인터록 여유(생산팀), 에너지 효율은 전력 사용량(환경안전 · 에너지팀), 현장 판단 지식 축적은 판단 선례 축적(공장 경영)으로 켜집니다. 지표마다 좋아지는 방향(오를수록 좋음 · 내릴수록 좋음), 다른 이름(예: 가동률을 가용성이라고도 부름), 단위, 재는 주기가 있습니다. 영업이익과 총비용에는 다른 지표로 계산하는 식이 적혀 있고, 인터록 여유에는 경고선과 위험선이 있습니다.

핵심은 지표끼리의 **영향 관계(INFLUENCES)**입니다. 관계마다 부호(같은 방향 · 반대 방향)와 강도(강함 · 중간 · 약함)가 붙습니다. 위로 올라가는 큰 줄기는 이렇습니다. 매출은 영업이익을 올리고 총비용은 내립니다. 보전비 · 에너지비 · 부품 구매비 · 재고 보관비 · 지체상금은 총비용을 올립니다. 재고량은 재고 보관비를, 전력 사용량은 에너지비를, 부품 단가는

부품 구매비를 올립니다. 설비 쪽 줄기는 「평균 고장 간격 → 설비 가동률 → 생산량 → 납기 준수율 · 매출」입니다. 납기 준수율은 지체상금을 내리고 브랜드 신뢰를 올리며, 브랜드 신뢰는 다시 매출을 올립니다. 인터록 여유와 판단 선례 축적은 가동률을 받치고, 작동유 잔여 수명은 평균 고장 간격을 늘리고 보전비를 줄이며, 평균 고장 간격도 보전비를 줄입니다. 품질 쪽 줄기는 「부품 품질 → 품질 클레임(반대 방향) → 브랜드 신뢰(반대 방향) · 총비용」이고, 부품 품질은 평균 고장 간격도 늘립니다. 「부품 단가가 내려가면 부품 품질도 내려간다」는 관계에는 **조건**이 붙어 있습니다. 공급사를 바꿔 단가를 낮출 때만 성립하고, 같은 부품의 시세가 변할 때는 해당하지 않습니다. 재고량과 매출 사이에는 일부러 선을 긋지 않았습니다. 직접 관계가 없기 때문입니다. 이 줄기들이 13장에서 「이 조치가 회사에 득인가 실인가」를 계산하는 길이 됩니다.

② **프로세스 층(BPMN 2.0 최소 집합)**. 일이 어떤 순서로, 누구에 의해, 어떤 데이터와 판단으로 진행되는지입니다.

프로세스는 「설비 이상 조치」(실행함), 「정비 작업지시」(설명용, 실행하지 않음), 「미지원 경보 현장 검토」(실행함)입니다. 각 프로세스는 이루려는 전략 목표를 가리킵니다(ACHIEVES). 설비 이상 조치는 설비 가용성 · 무재해 운전 · 납기 신뢰를, 정비 작업지시는 설비 가용성 · 원가 절감을, 미지원 경보 현장 검토는 무재해 운전을 이룹니다 (118). 프로세스는 설비를 대상으로 하고(ACTS_ON) 흐름 노드를 가집니다(HAS_NODE). 흐름 노드는 이벤트(시작 · 경계 · 종료), 작업(판단 작업 · 사용자 작업 · 서비스 작업), 게이트웨이(배타)이고, 순서 흐름(SEQUENCE_FLOW)으로 이어지며 흐름에는 조건 문장이 붙을 수 있습니다. 작업은 수행자를 가리킵니다(PERFORMED_BY, 역할이나 시스템) (113). 경계 타이머는 붙어 있는 작업을(ATTACHED_TO), 시작 이벤트는 자신이 받는 이상 패턴을(CORRELATES) 가리킵니다. 판단 작업은 자신이 부르는 판단을(INVOKES), 모든 작업은 읽는 데이터와 만드는 데이터를(READS · PRODUCES) 가리킵니다 (115). 「PLC 명령 발행」작업은 제어 스킬들을, 「정비 작업지시」작업은 작업지시 스킬들을 실행 대상으로 가리킵니다(EXECUTES). 지식 지도는 AI가 맡는 판단 작업을 「업무 규칙 작업」으로 적어 두었고, 실행 정의서(19장)는 같은 자리를 ProcessGPT 모양(사용자 작업 + 에이전트 완료 방식)으로 적습니다. 둘은 ⑦ 운영 기록 층의 실행 층에서 이어집니다.

③ **리소스 층(조직 · 시스템 · 설비, ISO 14224의 설비 분류 취지)**. 실제로 일하는 주체와 대상입니다. 부서는 공장 경영 · 생산팀 · 설비보전팀 · 구매팀 · 품질팀 · 영업팀 · 환경안전 · 에너지팀입니다. 역할은 부서에 속하고(MEMBER_OF) 승인 등급이 있습니다. 운전원(생산팀)이 가장 낮고, 생산관리자(생산팀) · 설비보전팀장(설비보전팀) · 구매팀장(구매팀) · 품질팀장(품질팀)이 같은 중간 등급, 공장장(공장 경영)이 가장 높습니다. 정보 시스템은 공장 구역의 SCADA · PLC(명령 게이트웨이 경유)와 사무실 구역의 기록 DB(Historian) · AI 에이전트 · HITL 프로세스 서비스 · CEP 탐지기(Flink 대체) · MES · ERP · CMMS · SCM 구매 포털 · QMS 품질 시스템입니다. 누가 어떤 스킬을 실행하는지도 이어져 있습니다 (HAS_SKILL). 제어 스킬은 SCADA가, 작업지시 스킬은 CMMS가 실행하고, 운전원과 설비보전팀장 역할에도 직접 다루는 스킬이 연결되어 있습니다.

설비 쪽은 「설비 → 구성품 → 센서 · 구동기 · 부품」의 나무입니다. 설비 HYD-01 · 02 · 03은 같은 구성품 형식을 공유합니다(HAS_COMPONENT). 오일 쿨러 · 쿨러 팬 · 주 펌프 · 예비 펌프 · 구동 모터 · 작동유 탱크입니다. 설비 노드에는 예측 모델의 이름과 적용 범위도 적혀 있습니다. 구성품은 센서로 감시되고(MONITORED_BY) 센서는 상태 변수를 관측합니다(OBSERVES). 유온 센서와 냉각 효율 가상 센서는 쿨러에, 토출 압력과 유량 센서는 주 펌프에, 팬 진동 센서는 팬에 붙어 있습니다. 구성품은 구동기로 움직이고(ACTUATED_BY) 구동기는 상태 변수를 조작합니다(MANIPULATES). 팬 인버터는 팬 속도를, 펌프 부하 설정은 펌프 부하를, 펌프 전환 밸브는 운전 펌프 선택을 바꿉니다.

상태 변수는 세 종류입니다. 사람이 바꾸는 **조작 변수**(팬 속도 · 펌프 부하 · 운전 펌프), 결과로 나타나는 **결과 변수**(유온 · 냉각 효율 · 토출 압력 · 유량 · 팬 진동, 일부에는 인터록 한계가 속성으로 붙음), 고장이 몰래 바꾸는 **외란 변수**(펌프 내부 누설 · 쿨러 핀 오염도 · 팬 베어링 마모도, 1장의 숨은 값)입니다.

물리 영향(INFLUENCES)이 이 변수들을 잇고, 끝에서 성과 지표로 넘어갑니다. 팬을 올리면 유온이 내려가고 진동은 커지며(베어링이 닳았을수록 크게) 전력이 듭니다. 부하를 올리면 발열이 커져 유온이 오르고 압력 · 생산량 · 전력 사용량이 오릅니다. 냉각 효율이 오르면 유온이 내려가고, 핀 오염은 냉각 효율을 떨어뜨립니다. 누설은 압력과 유량을 떨어뜨리고 유온을 조금 올립니다. 베어링 마모는 진동을 키웁니다. 지표로 넘어가는 다리도 있습니다. 유온은 작동유 잔여 수명(고온에서 크게)과 인터록 여유를 깎고, 진동은 평균 고장 간격과 설비 가동률(진동 인터록)을 깎습니다. 압력과 유량이 떨어지면 생산량이 줄고(사이클 지연), 압력이 크게 떨어지면 가동률이 깎입니다(저압 인터록). 「유량 → 생산량」은 전문가 질문의 답으로 나중에 보강된 관계입니다. 부품은 구성품에 쓰이고(USES_PART) 쿨러 코어 · 펌프 축 씌 키트 · 팬 베어링이 있습니다. 공급사는 A정밀(저가, 승인), B-OEM(순정, 승인), C트레이딩(최저가, 비승인)이고, 부품과 공급사 사이의 견적 관계(SUPPLIED_BY)에 가격 · 불량률 · 납기 일수가 붙습니다. 공급사 노드의 「승인 공급사 목록(AVL)」여부가 구매 규칙의 근거가 됩니다.

9장 지식 지도 ②: 진단 · 스킬과 규칙 · 외부 · 운영 기록

④ **설비 진단 총(ISO 13374 / MIMOSA OSA-CBM의 상태 감지 · 건강 평가 · 권고)**. 사슬 하나로 읽힙니다. 이상 패턴 → 증상 → 고장 유형 → 원인 → 증거. **이상 패턴**은 이름 · 코드 · 조건식 · 유지 시간을 가지고, 어떤 입력 데이터를 어떤 비교로 보는지(TESTS)와 근거 매뉴얼 절(DERIVED_FROM)을 가리킵니다. 쿨러 성능 저하 · 펌프 내부 누설 · 팬 진동 상승 · 과열 인터록 트립 · 저압 인터록 트립 · 고진동 인터록 트립 · 오일 분석 기준 이탈(사람 입력)이 있습니다. 패턴은 증상을 찾아내고(DETECTS), 증상은 센서로 관측됩니다(OBSERVED_BY). 증상은 유온 상승 · 냉각 효율 저하 · 토출 압력 저하 · 유량 저하 · 진동 증가 · 오일 분석 기준 이탈입니다. 증상은 고장 유형을 가리키고(INDICATES), 고장 유형은 구성품에서 일어납니다(OCCURS_IN) (112). 쿨러 냉각 성능 상실(쿨러) · 펌프 체적 효율 저하(주 펌프) · 팬 베어링 열화(팬) · 과열 정지(구동 모터) · 작동유 열화(작동유 탱크) · 저압 정지(주 펌프) · 고진동 정지(팬)입니다.

고장끼리 번지는 길(LEADS_TO)이 있습니다. 냉각 성능 상실은 과열 정지와 작동유 열화로, 펌프 체적 효율 저하는 작동유 열화와 저압 정지로, 팬 베어링 열화는 냉각 성능 상실과 고진동 정지로 번집니다. 그래서 「팬 베어링이 닳으면 팬을 늦춰야 하고, 냉각이 줄어 과열 정지로 갈 수 있다」 같은 연쇄를 그래프로 따라갈 수 있습니다. 세 가지 정지(트립)에는 원인이 따로 없습니다. 앞선 고장의 결과이기 때문입니다. **원인**은 고장 유형을 일으키고(CAUSES) 사전확률을 가지며, 관련 부품(INVOLVES_PART)과 몰래 바꾸는 외란 변수(DISTURBS)를 가리킵니다. 냉각 성능 상실의 원인은 쿨러 핀 오염(쿨러 코어 · 핀 오염도)과 주변 온도 상승, 펌프 체적 효율 저하의 원인은 펌프 축 씌 마모(씌 키트 · 누설), 팬 베어링 열화의 원인은 팬 베어링 마모(팬 베어링 · 베어링 마모도), 작동유 열화의 원인은 고온 운전에 의한 산화 · 수분 혼입 · 오염 입자 유입 · 교환 주기 초과입니다.

증거 규칙(EVIDENCED_BY)은 원인을 지지하는 관측입니다. 규칙마다 이름, 무엇을 어떤 비교로 보는지, 가중치, 그리고 기록 DB에 던질 질의문과 읽을 태그 · 구간이 노드 안에 들어 있습니다. 조사관은 이 질의문을 그대로 실행할 뿐 스스로 만들지 않습니다. 쿨러 핀 오염에는 「냉각 효율 평균이 낮은가」와 「팬 진동이 정상인가(팬 고장이 아닌가)」, 주변 온도

상승에는 「캐비닛 온도가 높은가」, 축 썰 마모에는 「토출 압력 평균이 낮은가」와 「유량 평균이 낮은가」, 베어링 마모에는 「진동이 최근 크게 올랐나」와 「유온이 정상인가(냉각 문제가 아닌가)」가 붙어 있습니다. 작동유 열화의 원인들에는 증거 규칙이 없습니다. 실시간 센서가 없고 사람이 넣는 오일 분석 결과로만 판단하기 때문입니다.

지식 출처는 HYD 유압 파워팩 운전 · 정비 매뉴얼, 설비안전규정, 구매규정, 가격 정책, 경남유압 BSC 전략맵이고 모두 교육용 예시입니다. 매뉴얼은 절로 쪼개져 매뉴얼에 속합니다(PART_OF). 절은 「정비를 위한 현장 전환」 · 「운전 모드와 원격 조치 권한」 · 「예비 펌프 전환」 · 「압력 설정 변경 금지」 · 「펌프 축 썰 교체」 · 「팬 증속 운전」 · 「냉각 회복 판정」 · 「쿨러 핀 세척」 · 「캐비닛 환기」 · 「팬 진동 점검」 · 「팬 베어링 교체」 · 「부하 저감 운전」 · 「트립 후 재기동」이고, 절마다 요지 문장이 붙어 있습니다. 패턴 · SOP 단계 · 판단 규칙이 이 절들을 인용합니다. 검사관이 읽는 탐지 정의도 이 층에 패턴과 함께 붙어 있습니다.

⑤ **스킬 · 규칙 층(SOP와 DMN 1.x 최소 집합)**. 앞 절반은 조치 방법입니다. 스킬 하나가 SOP 하나이고, SOP 번호와 종류(제어 · 작업지시)를 가집니다. 고장 유형은 스킬에 「즉시 완화」(MITIGATED_BY)나 「근본 조치」(REMEDIED_BY)로 이어지고 (111), 어떤 스킬은 특정 원인일 때만 맞습니다(ADDRESSES). 스킬은 승인할 역할을 가리키고(APPROVED_BY) (117), 단계(HAS_STEP)를 가지며 단계마다 근거 매뉴얼 절을 인용합니다(REFERS_TO). 스킬은 원자 조치로 쪼개지고(CONSISTS_OF, 순서와 값), 원자 조치는 무엇을 움직이는지를 가리킵니다(TARGETS). 팬 속도 설정은 팬 인버터, 펌프 부하 설정은 부하 설정 장치, 운전 펌프 전환은 펌프 전환 밸브를 가리키고, 압력 설정 상향(이 PLC에는 없음) · 계획 정지 · 인터록 리셋도 펌프 구동 쪽을 가리키는 **설비 명령**입니다. 정비 작업지시 발행은 CMMS, 부품 구매요청은 ERP를 가리키는 **회사 거래**입니다. 마지막으로 스킬은 자신이 움직이는 상태 변수나 성과 지표를 가리킵니다(AFFECTS, 부호와 변화량, 메모). 이것이 가치 층으로 가는 입구입니다 (110). SOP 하나하나가 어느 고장에 어떻게 붙는지는 10장에서 사슬로 따라갑니다.

뒤 절반은 판단입니다. 판단(Decision)은 결정표(DecisionTable, 적중 방식 포함)로 구현되고(IMPLEMENTED_BY), 결정표는 규칙(Rule)을 행으로 가집니다(HAS_RULE). 규칙에는 순서 · 조건식 · 효과(고름 SELECT · 제외 EXCLUDE · 경고 WARN · 감점 PENALTY · 순위 RANK) · 설명이 있습니다. 그리고 어떤 입력을 어떤 비교로 보는지(TESTS), 고르는 규칙이면 무엇을 내놓는지(OUTPUTS, 원인이나 스킬), 거르는 규칙이면 어느 스킬에 적용되는지(APPLIES_TO, 없으면 모든 후보), 감점 규칙이면 어느 지표 때문인지(PENALIZES), 근거가 무엇인지(DERIVED_FROM, 매뉴얼 절이나 규정)를 가리킵니다. 판단은 필요한 입력(REQUIRES_INPUT), 앞선 판단(REQUIRES_DECISION), 따르는 지식 출처(GOVERNED_BY)를 가리킵니다. **입력 데이터**(InputData)는 판단과 작업 사이를 흐르는 값의 정의로, 변수 이름 · 형식을 가지고 출처(SOURCED_FROM, 센서나 시스템)와 그것이 나타내는 상태 변수 · 지표(REPRESENTS)를 가리킵니다. 판단은 네 단계 사슬(원인 판정 → 조치 후보 → 규정 적합성 → 조치 우선순위)이고, 내용은 11장에서 봅니다.

⑥ **외부 변수 · 예측 층**. 회사가 통제하지 못하지만 영향을 주는 변수와 그 출처입니다. 원/달러 환율(한국은행) · 외기 온도(기상청) · 전력 단가(한전) · 고객 수요(영업 수요예측). 이것들도 영향 관계(INFLUENCES)로 지표나 상태 변수에 이어집니다 (109). 환율은 부품 단가를 올리고, 가격 정책에 따라 조건부로 매출을 깎거나(원가 상승을 판가에 반영하지 못할 때) 재고를 늘립니다(판가를 올려 판매가 줄 때). 외기 온도는 유온을, 전력 단가는 에너지비를 올리고, 고객 수요는 매출을 올리고 재고를 줄입니다. 그리고 조치별 **예측** 노드가 있습니다. 예측은 어떤 원인을 전제로(GIVEN), 어떤 스킬을 가정했을 때(ASSUMES), 어떤 상태 변수가(FORECASTS) 어디서 멈추는지를 적은 값이고, 계산 방법(열 평형 모델 또는 설계 목표값)이 함께 적혀 있습니다. 이 저장값은 사람이 보는 참고용입니다. 판단 엔진은 저장값을 쓰지 않고 매번 설비의 최신 상태로 예측 모델을 새로 돌립니다(13장).

⑦ **운영 기록 총**. 실행 중에 쌓이는 것입니다. **사건(Incident)**은 설비(ON_ASSET) · 일으킨 패턴(RAISED_BY) · 판정 원인(DIAGNOSED_AS)을 가리키고 (119), **판단 사례(DecisionCase)**는 그 사건(FOR_INCIDENT) · 고른 스킴(CHOSE) · 결정한 역할(DECIDED_BY) · 어떤 판단의 실행이었는지(INSTANCE_OF, 조치 우선순위)를 가리키며 권고를 따랐는지와 사유 · 시각을 담습니다 (116). 이것이 다음 판단의 선례가 됩니다. 처음부터 예시 선례가 몇 건 들어 있습니다. 「OEM 납기 우선, 야간 세척 예정」 이라며 팬 최대와 부하 낮춤을 고른 사례, 「이번 주 팬 최대 운전이 이미 길었다」 며 권고와 다르게 부하 낮춤과 야간 세척을 고른 사례, 「예비 펌프 정비 완료 상태 확인」 뒤 예비 펌프 전환을 고른 사례입니다. 스키마에는 기술 기록 클래스들도 있습니다. 운영 중에는 업무 진행이 이 층에 더 쏩니다. 인스턴스 방식의 실행 층(정의 판본 · 판본의 흐름 노드 · 프로세스 인스턴스 · 작업 노드와 그 관계, 19장), 투영이 순서를 지키게 하는 울타리 노드, 지식 편집 영수증, DDL 적재 묶음, 매뉴얼 문서와 적재 묶음입니다. 실행 층의 인스턴스는 지식 지도의 프로세스를, 판본의 흐름 노드는 프로세스 층의 같은 의미 노드를 가리킵니다(INSTANCE_OF · MAPS_TO) (114).

10장 지식 지도 ③: 고장마다 사슬 따라가기

온톨로지는 따로따로 외우면 표 묶음이지만, 경보 하나에서 출발해 관계를 따라가면 하나의 이야기가 됩니다. 조사관과 판단 엔진이 실제로 따라가는 순서대로 고장마다 사슬을 걸어 보겠습니다. 순서는 늘 같습니다. 패턴 → 증상 → 고장 유형 (구성품) → 번지는 고장 → 원인(부품 · 외란 변수) → 증거 규칙 → SOP(즉시 완화 · 근본 조치 · 원인 한정) → 원자 조치 (설비 명령 · 회사 거래) → 움직이는 상태 변수 · 지표 → BSC 경로 → 승인 역할 → 매뉴얼 절.

쿨러 성능 저하. 패턴 「쿨러 성능 저하」는 유온 · 냉각 효율 · 유온 기울기를 보고(근거 「팬 증속 운전」 절), 증상 「유온 상승」과 「냉각 효율 저하」를 찾아냅니다. 두 증상은 유온 센서와 냉각 효율 가상 센서로 관측되고, 고장 유형 「쿨러 냉각 성능 상실」(오일 쿨러에서 일어남)을 가리킵니다. 이 고장은 「과열 정지」와 「작동유 열화」로 번집니다. 원인 후보는 「쿨러 핀 오염」(부품 쿨러 코어, 외란 변수 핀 오염도)과 「주변 온도 상승」입니다. 핀 오염은 냉각 효율 평균과 팬 진동 정상 여부로, 주변 온도 상승은 캐비닛 온도로 확인합니다. 즉시 완화(MITIGATED_BY)로 「팬 최대」(제어, 운전원 승인), 「팬 최대와 부하 낮춤」(제어, 생산관리자), 「부하 낮춤과 야간 세척」(제어, 생산관리자, 핀 오염일 때만), 근본 조치(REMEDIED_BY)로 「쿨러 핀 세척」(작업지시, 설비보전팀장, 핀 오염일 때만)과 「캐비닛 환기 개선」(작업지시, 설비보전팀장, 주변 온도 상승일 때만)이 붙어 있습니다.

원자 조치로 보면 「팬 최대」는 팬 속도 설정 하나, 「팬 최대와 부하 낮춤」은 팬 속도 설정과 펌프 부하 설정, 「부하 낮춤과 야간 세척」은 펌프 부하 설정과 쿨러 핀 세척 작업지시(회사 거래)로 이루어집니다. 핀 세척과 환기 개선은 CMMS 작업지시 하나씩입니다. 움직이는 것(AFFECTS)은 팬 속도 · 펌프 부하 · 핀 오염도 · 유온 · 보전비이고, 여기서 가치 층으로 올라갑니다. 팬을 올리면 「팬 속도 → 유온(내려감) → 인터록 여유 · 작동유 수명(좋아짐) → 가동률 · 평균 고장 간격」으로 득이 생기지만, 「팬 속도 → 진동 → 평균 고장 간격 · 가동률」과 「팬 속도 → 전력 사용량 → 에너지비 → 총비용」으로 실도 생깁니다. 부하를 낮추면 「펌프 부하 → 생산량(줄어듦) → 납기 준수율 · 매출」로 실이, 「펌프 부하 → 유온 · 전력 사용량」으로 득이 생깁니다. 작업지시는 「보전비 → 총비용」의 실과 「핀 오염도 → 냉각 효율 → 유온」의 득을 함께 가집니다. 단계가 인용하는 절은 「운전 모드와 원격 조치 권한」 · 「팬 증속 운전」 · 「부하 저감 운전」 · 「냉각 회복 판정」 · 「정비를 위한 현장 전환」 · 「쿨러 핀 세척」 · 「캐비닛 환기」입니다.

펌프 내부 누설. 패턴 「펌프 내부 누설」은 PLC 운전 상태 · 토출 압력 · 유량 · 부하 설정을 보고(근거 「예비 펌프 전환」 절), 증상 「토출 압력 저하」 · 「유량 저하」(주 펌프의 토출 압력 · 유량 센서)를 거쳐 고장 유형 「펌프 체적 효율 저하」(주 펌프)를 가리킵니다. 이 고장은 「작동유 열화」와 「저압 정지」로 번집니다. 원인은 「펌프 축 씰 마모」(부품 펌프 축 씰 키트, 외란 변수 누설)이고, 증거는 토출 압력 평균과 유량 평균입니다. 즉시 완화는 「예비 펌프 전환」(제어, 생산관리자, 원자 조치는 운전 펌프 전환), 「부하 낮춤」(제어, 운전원), 「압력 설정 상향」(제어, 운전원, 개정 매뉴얼이 누설 때 금지한 옛 절차라 규칙이 제외함)이고, 근본 조치는 「펌프 축 씰 교체」(작업지시, 설비보전팀장, 씰 마모일 때만)입니다. 씰 교체의 원자 조치는 순정 공급사에 내는 부품 구매요청(ERP)과 교체 작업지시(CMMS) 두 거래입니다. 예비 펌프 전환은 누설을 없애지만 「예비 설비 여유가 사라진다」며 평균 고장 간격을 깎는 관계를 가지고, 압력 상향은 압력을 올리는 대신 누설을 키우는 관계를 가집니다. 씰 교체 단계는 「정비를 위한 현장 전환」 · 「펌프 축 씰 교체」를, 전환과 부하 낮춤 단계는 「운전 모드와 원격 조치 권한」 · 「예비 펌프 전환」 · 「부하 저감 운전」을, 압력 상향 단계는 「압력 설정 변경 금지」를 인용합니다. 구매요청 쪽에는 「부품 → 공급사 견적 → 공급사 승인 여부」가 이어져 있어 구매규정 규칙이 여기서 걸립니다.

팬 진동 상승. 패턴 「팬 진동 상승」은 진동과 진동 기울기를 보고(근거 「팬 진동 점검」 절), 증상 「진동 증가」(팬 진동 센서)를 거쳐 「팬 베어링 열화」(팬)를 가리킵니다. 이 고장은 「쿨러 냉각 성능 상실」과 「고진동 정지」로 번집니다. 원인은 「팬 베어링 마모」(부품 팬 베어링, 외란 변수 베어링 마모도), 증거는 진동이 최근 크게 올랐는가와 유온이 정상인가입니다. 즉시 완화는 「팬 낮춤과 부하 낮춤」(제어, 생산관리자)과 「팬 낮춤」(제어, 운전원)이고, 근본 조치는 「계획 정지와 베어링 교체」(공장장 승인, 베어링 마모일 때만, 원자 조치는 계획 정지 명령과 베어링 교체 작업지시)와 「팬 베어링 교체」(작업지시, 설비보전팀장, 베어링 마모일 때만)입니다. 계획 정지는 가동률을 깎고 보전비를 올리는 대신 베어링 마모를 없애는 관계를 가집니다. 이 고장의 맛바꿈이 지식에 그대로 드러납니다. 팬을 낮추면 「팬 속도 → 진동(내려감)」은 득이지만 「팬 속도 → 유온(오름) → 인터록 여유 · 작동유 수명」은 실이고, 그래서 낮춤과 함께 부하를 낮추는 안이 따로 있습니다. 단계는 「팬 진동 점검」 · 「부하 저감 운전」 · 「냉각 회복 판정」 · 「팬 베어링 교체」 · 「정비를 위한 현장 전환」을 인용합니다.

세 가지 트립. 과열 · 저압 · 고진동 인터록 트립 패턴은 PLC 상태가 트립인지를 보고(근거 「트립 후 재기동」 절), 각각 증상 유온 상승 · 토출 압력 저하 · 진동 증가를 거쳐 고장 유형 「과열 정지」(구동 모터) · 「저압 정지」(주 펌프) · 「고진동 정지」(팬)를 가리킵니다. 이 고장들은 원인이 없고, 앞선 고장(냉각 성능 상실 · 체적 효율 저하 · 베어링 열화)에서 번져 온 끝입니다. 그래서 증거 규칙도 없습니다. 각 정지에는 즉시 완화로 리셋 SOP가 하나씩 붙습니다. 과열 정지에는 「냉각 후 리셋」, 저압 정지에는 「압력 복귀 후 리셋」, 고진동 정지에는 「진동 확인 후 리셋」이고, 모두 제어 스킬 · 원자 조치는 인터록 리셋 · 생산관리자 승인입니다. 단계는 「원인 조치가 끝났는가 → 값이 돌아왔는가 → 리셋하고 재관측」의 같은 모양이고, 「트립 후 재기동」절과 함께 압력이면 「예비 펌프 전환」, 진동이면 「팬 진동 점검」절을 인용합니다. 세 리셋 모두 재기동으로 가동률을 올리는 관계를 가집니다.

오일 분석 이탈. 패턴 「오일 분석 기준 이탈」은 센서가 아니라 사람이 넣는 정기 오일 분석 결과(점도 · 산가 · 수분 · 청정도)를 입력 데이터로 보고, 그 입력의 출처는 CMMS로 적혀 있습니다. 증상 「오일 분석 기준 이탈」은 「작동유 열화」(작동유 탱크)를 가리키고, 원인은 고온 운전에 의한 산화 · 수분 혼입 · 오염 입자 유입 · 교환 주기 초과입니다. 원인 판정 규칙은 산화를 가장 앞에 두고 나머지는 분석 항목으로 구분합니다. 작동유 열화의 조치 후보 규칙도 있지만, 그 규칙이

내놓을 스킬(작동유 교환 · 열화 점검)은 해당 매뉴얼 장을 적재해야 생깁니다. 작동유 열화는 냉각 성능 상실과 체적 효율 저하에서 먼저 오는 곳이기도 해서, 쿨러나 펌프 문제를 오래 방치하면 결국 오일 문제로 이어진다는 길이 지식에 남아 있습니다. 실제로 이 패턴을 탐지하는 코드는 없습니다.

BSC 경로를 따라가는 방법. 판단 엔진은 스킬에서 시작해 「스킬 → AFFECTS → 상태 변수나 지표 → INFLUENCES → ... → 지표」로 영향 관계를 몇 단계까지 따라가며, 지나는 관계의 부호로 방향을, 강도를 곱해 크기를 정합니다. 같은 지표 · 같은 방향에 여러 길로 닿으면 가장 큰 것만 씁니다. 조건이 붙은 관계(공급사를 바꿀 때만, 가격 정책에 따라)를 지나면 그 특성은 「조건부」로 따로 씁니다. 이렇게 모인 「어느 지표가 좋아지고 어느 지표가 나빠지는가」 목록이 카드의 BSC 특성이 되고, 포털 카드에 경로째 보입니다.

승인 역할은 조직과 이어집니다. 카드의 승인 역할은 스킬의 APPROVED_BY에서 오고, 그 역할은 부서에 속하며 등급을 가집니다. 그래서 「이 카드는 생산관리자 이상」이라는 판정이 조직도 그래프에서 그대로 나옵니다. 사람이 지식 화면에서 승인 역할을 바꾸면(23장) 다음 판단부터 바뀐 역할이 적용됩니다.

11장 판단 정의 넷: DMN으로 본 판단

판단 엔진이 하는 일은 지식 지도에 「판단」으로 선언되어 있습니다. 판단은 사슬입니다. 조치 후보는 원인 판정을, 규정 적합성은 조치 후보를, 조치 우선순위는 규정 적합성을 앞선 판단으로 요구합니다(REQUIRES_DECISION). 판단마다 질문 문장 · 결정표와 적중 방식 · 필요한 입력 · 따르는 지식 출처가 있습니다.

원인 판정. 질문은 「이 경보의 고장 유형과 근본 원인은 무엇인가?」. 결정표의 적중 방식은 PRIORITY(맞는 규칙 가운데 우선순위가 가장 높은 하나), 입력은 경보 패턴(출처 CEP 탐지기)과 유온 · 냉각 효율 · 토출 압력(출처 각 센서), 지식 출처는 매뉴얼입니다. 규칙은 고장마다 있습니다. 「쿨러 패턴이고 냉각 효율이 낮으면 쿨러 핀 오염」(근거 「팬 증속 운전」 절), 「누설 패턴이고 토출 압력이 낮으면 축 쉴 마모」(「예비 펌프 전환」 절), 「진동 패턴이고 유온이 정상이면 베어링 마모」(「팬 진동 점검」 절), 「오일 분석 패턴이면 산화를 먼저, 나머지는 분석 항목으로」(「트립 후 재기동」 절). 그런데 이 결정표는 **선언만 되어 있고 실행되지 않습니다**. 실제 원인 순위는 조사관이 「사전확률 × 맞은 증거의 비중」으로 계산합니다(12장). 출력은 판정 원인과 판정 고장 유형이고, 둘 다 「AI 에이전트」를 출처로 하는 입력 데이터가 되어 다음 판단으로 넘어갑니다.

조치 후보. 질문은 「이 고장 유형에 쓸 수 있는 조치 방법은 무엇인가? 원인 한정 스킬은 원인으로 거른다」. 적중 방식은 COLLECT(맞는 규칙을 모두 모음), 입력은 판정 고장 유형 · 판정 원인 · PLC 상태(출처 SCADA) · 경보 패턴입니다. 규칙은 모두 「고름」입니다. 고장 유형별 규칙(냉각 성능 상실 · 체적 효율 저하 · 베어링 열화)이 각 고장의 즉시 완화와 근본 조치 스킬을 내놓고(OUTPUTS), 트립 규칙은 두 모양입니다. 「PLC가 트립이고 경보가 그 트립 패턴이면」과 「판정 고장 유형이 그 정지면」 각각에서 해당 리셋 스킬을 내놓습니다. 작동유 열화 규칙은 매뉴얼 적재 뒤에 채워집니다. 원인 한정 (ADDRESSES)은 규칙 행이 아니라 엔진이 지식 관계를 보고 거릅니다. 출력은 후보 스킬 목록입니다.

규정 적합성. 질문은 「후보 스킬 중 규정상 쓸 수 없거나 감점할 것은 무엇인가?」. 적중 방식은 COLLECT이고 입력이 가장 많습니다. 후보 스킬 종류와 명령 코드, 조치 후 예측 유온과 예측 토출 압력(출처 AI 에이전트), 판정 고장 유형, PLC 운전 모드와 상태(SCADA), 팬 최대 연속 운전 시간(기록 DB), 예비 펌프 가용(CMMS), 공급사 승인 여부(SCM). 지식 출처는 설비안전규정 · 매뉴얼 · 구매규정입니다. 규칙은 효과별로 이렇습니다. **제외**는 「누설 의심 때 압력 설정 상향

금지」(「압력 설정 변경 금지」 절), 「예측 유온이 인터록 한계에 닿음」(설비안전규정, 모든 후보), 「원격 제어는 원격 자동 모드에서만」(「운전 모드와 원격 조치 권한」 절, 제어 스킬들), 「예비 펌프가 정비 중이면 전환 불가」(「예비 펌프 전환」 절), 「핵심 부품은 승인 공급사에서만 구매」(구매규정, 축 싺 교체), 「트립 중에는 리셋 외 제어 명령 불가」(「트립 후 재기동」 절, 모든 후보). **감점**은 「팬 최대 연속 운전이 길면 수명 감점」(「팬 증속 운전」 절, 팬 최대가 든 스킬들, 감점 대상 지표는 평균 고장 간격). **경고**는 「예측 유온이 경보 해제선에 못 미칠 수 있음」(「냉각 회복 판정」 절, 팬만 조절하는 스킬들)과 「예측 토출 압력이 낮아 저압 경보가 이어질 수 있음」(「예비 펌프 전환」 절, 모든 후보). 규칙이 묻는 사실을 모르면 경고는 붙이고 제외 · 감점은 제외로 칩니다. 출력은 후보마다 「쓸 수 있는가 · 제외 사유 · 감점 · 경고」입니다.

조치 우선순위. 질문은 「남은 후보 중 회사 가치(영업이익)에 가장 유리한 순서는?」. 적중 방식은 UNIQUE(규칙 하나), 입력은 예측 유온 · 긴급 오더 남은 시간(MES) · 납기 지연 보상과 고객 등급(ERP) · 고온 구간 출하 대기 로트의 클레임 위험과 수량(QMS)이고, 지식 출처는 BSC 전략맵입니다. 규칙은 「순위」 하나이고, 그 안에 검토를 거친 점수식(순위 정책)이 속성으로 들어 있습니다. 점수식은 구성 요소의 합입니다. BSC 득실(조건부는 절반만), 예측 유온 여유, 경고, 감점, 선례 비율, 납기 긴급도와 생산 영향(유지 · 감산 · 정지)의 곱, 출하 품질 위험. 어떤 업무 입력을 어떤 이름으로 읽을지도 정책 안에 별칭으로 적혀 있어서, 새 업무 조건은 코드가 아니라 DDL 등록과 정책 변경으로 늘어납니다. 동점이면 낮은 등급이 승인할 수 있는 쪽이 앞입니다.

실행되는 것과 선언만 된 것. 판단 엔진은 돌리기 전에 지식 지도의 선언을 자기 실행 방식과 맞춰 봅니다. 엔진이 돌리는 적중 방식은 조치 후보 COLLECT · 규정 적합성 COLLECT · 우선순위 UNIQUE이고, 선언이 다르면 판단을 거부합니다. 원인 판정의 PRIORITY 결정표는 이 비교에서 빠지고 실행되지 않습니다. 규칙이 검사하는 입력은 반드시 그 판단의 필요한 입력으로 선언되어 있어야 합니다. 출력의 흐름은 이렇습니다. 원인 판정(조사관) → 판정 원인 · 고장 유형 → 조치 후보 → 후보 스킬 목록 → 규정 적합성(후보마다 예측을 먼저 계산) → 후보별 판정 → 우선순위 → 순위가 매겨진 카드 묶음과 권고한 장 → 카드 검사 → 업무 진행에 제출. 카드 한 장에는 그 카드를 고른 규칙, 걸린 규칙과 근거 절, 예측값과 예측 조건, BSC 득실 경로, 선례, 승인 역할, SOP 단계가 인용으로 남습니다. 인스턴스 방식에서는 이 네 판단이 BPMN의 AI 작업과 하나씩 대응합니다(INVOKEs).

12장 AI 조사관: 원인 진단

기본 방식에서는 경보(울림)가 오면 AI 조사관(agent)이 장부에서 그 경보를 읽고 깨어납니다 (26). 순서는 이렇습니다. ① 업무 진행에게 「이 패턴을 이 흐름으로 처리하는가」(경보 정책)를 묻고, 아니면 보류합니다(트립 · 오일 분석). ② 데이터가 신선한지 봅니다. 기록 DB의 최신 TS1이 묵었거나 수집 다리가 멈췄으면(HTTP로 상태 확인) (32) 추론을 보류합니다.

③ 조사관의 도구들(이름에 mcp가 붙어 있지만 별도 서버가 아니라 같은 프로그램 안의 함수)이 (28) 지식 지도에서 미리 정해 둔 질의(Cypher 템플릿)만 읽기 전용으로 실행해 경보 패턴 → 증상 → 고장 유형 → 원인 후보와 각 원인의 증거 규칙을 꺼냅니다(Bolt) (29). 지식 지도에는 원인 판정이 결정표(적중 방식 PRIORITY)로 선언되어 있지만, 조사관이 실제로 쓰는 순위는 「사전확률 × 맞은 증거의 비중」입니다.

④ 후보마다 증거 질문을 기록 DB에 시계열 읽기 전용 계정으로 던집니다(SQL) (30). 쿨러 핀 오염에는 「냉각 효율이 낮은가」 「팬 진동이 정상인가(팬 고장이 아닌가)」, 주변 온도 상승에는 「캐비닛 온도가 높은가」. 그런데 캐비닛 온도는 열 모델에서 사실상 변하지 않아, 이 증거는 늘 「아니다」 이고 주변 온도 상승은 이 설비에서 지지되지 않습니다. 확인할 수 없는 증거(값 없음 · 관측 공백 · 질의 오류)는 「아니다」로 세지 않고 「모름」으로 두며, 모름이 있으면 카드를 보류합니다. 모든 원인의 증거가 「아니다」면 「근거 없음」으로 보류합니다. 이 경보에서는 쿨러 핀 오염이 1순위가 됩니다.

데이터: 안내 카드. ⑤ 1순위 원인에 연결된 SOP와 단계, 매뉴얼 절을 붙여 안내 카드를 씁니다. 카드에는 경보 · 원인 후보와 증거 판정 · 권고 조치 · 데이터 신선도 · 인용한 지식 지도 노드 목록이 들어 있습니다. ⑥ 언어 모델이 카드의 일부만 보고 한국어 요약 문장을 새로 씁니다(HTTPS) (34). 키가 없거나 실패하면 정해진 문장을 씁니다. 언어 모델은 설정으로 고릅니다. 「자동」이면 OpenAI 키가 있을 때 OpenAI 호환, 없으면 Anthropic이고, OpenAI 호환 주소를 바꾸면 강사의 GPU 서버(SGLang)나 중계(LiteLLM)도 쓸 수 있습니다.

⑦ 가드레일이 카드를 검사합니다 (27). 증거 판정이 보류되지 않았는가, 인용이 있는가, 원인 후보가 있는가, 데이터가 신선한가, 명령 번호 · 쓰기 값 · 만료 시각 · MQTT 칸 이름처럼 AI가 쓰면 안 되는 항목이 없는가, 원인 · 증거 · 조치가 모두 지식 지도 노드를 인용하는가, 모르는 증거가 없는가, 조치 값이 허용 범위 안인가. 요약 문장 자체는 검사하지 않습니다. ⑧ 통과한 카드를 업무 진행에 제출하면 사건이 생기고 「승인 대기」가 됩니다(HTTP) (35). 조사관의 실행 기록은 메모리에만 있어 재시작하면 사라집니다.

조사관 안의 흐름. 한 경보에 대한 조사관의 실행은 단계 기록으로 남습니다. 경보 정책(업무 진행에게 물은 지원 범위와 사람 검토 경로) → 신선도(최신 유온의 나이와 수집 다리 상태) → 원인 후보(템플릿 질의로 꺼낸 원인 · 사전확률 · 고장 유형 · 증거 목록) → 증거(증거 규칙의 질의문 실행 결과) → 증거 판정(모름이나 근거 없음이면 여기서 보류) → 원인 순위 → 스킬(가장 앞선 원인의 고장 유형에 이어진 SOP와 관계 종류) → 카드(요약 문장 · 출처 · 인용) → 가드레일(위반 목록) → 판단 엔진의 카드 → 제출. 각 단계는 입력 · 출력 · 메모를 가지고 실행 기록에 쌓이며 (99), 포털의 「이상 확인 · 조치」 화면이 이 기록을 불러 단계별 근거로 보여 줍니다 (77). 같은 경보 번호로 올림이 다시 와도 실행을 새로 만들지 않습니다. 실행 기록은 메모리에 있어 조사관이 재시작하면 사라지지만, 제출된 카드는 업무 진행에 남습니다. 조사관은 같은 읽기 통로로 지식 지도 그래프 · 질의 템플릿 · 역할 · 패턴 목록도 포털에 내줍니다(22장).

13장 판단 엔진: 후보 → 규정 → 순위

이어서 판단 엔진(언어 모델 없음, 같은 입력이면 늘 같은 답)이 나머지 세 판단을 사슬처럼 푼다 (33). 엔진이 실제로 돌리는 적중 방식은 조치 후보 COLLECT(맞는 규칙을 모두 모음) · 규정 적합성 COLLECT · 우선순위 UNIQUE(점수식 하나)이고, 지식 지도의 선언이 이것과 다르면 판단을 거부합니다. 판단 엔진이 실패해도 사건은 카드 없이 승인 대기로 남습니다.

입력이 어디서 오는가. 지식 지도에는 판단에 쓰는 「입력 데이터」마다 출처(센서 · 검사관 · SCADA · 기록 DB · MES · ERP · CMMS · QMS · SCM · AI 에이전트 · 업무 진행)가 적혀 있고, 엔진은 그 출처대로 읽습니다. 센서 값과 팬 연속 운전 시간은 기록 DB에서(SQL), PLC 모드 · 상태와 예측에 쓸 설비 상태는 업무 진행이 아는 최신 설비 상태에서(HTTP), 회사 숫자(MES의 긴급 오더 남은 시간, ERP의 자연 보상과 고객 등급, QMS의 출하 대기 로트와 클레임 위험, CMMS의 예비

펌프 가용)는 회사 시스템 목업에서(HTTP) (31), 공급사 승인 여부는 지식 지도의 공급사 노드에서(Bolt) 읽습니다. 업무 DB의 특정 표 · 칸에 직접 연결해 둔 입력(23장의 DDL 등록으로 만든 「물리 입력」)은 업무 기록부의 기업 데이터에서 읽기 전용 계정으로 바로 읽습니다(SQL) (61).

예측. 조치마다 「이 조치를 하면 기름 온도 · 압력 · 진동이 어디서 멈추나」를 설비 예측 모델(설비 열 모델과 같은 구조)로 매번 계산합니다. 입력은 업무 진행이 받은 최근 설비 상태이고, 고장이 아직 진행 중이면(숨은 값이 변하는 중) 예측하지 않습니다. 예측할 수 없는 후보, 그리고 예측값이 인터록에 닿는 후보는 규칙과 별도로 제외됩니다.

조치 후보는 판정된 고장 유형에 이어진 스킬을 모읍니다. 쿨러 열화면 즉시 완화들과 근본 조치 작업지시들이 후보가 되고, 원인에 한정된 스킬은 그 원인일 때만 남습니다(핀 오염이면 바깥 고온 전용 환기 작업은 빠짐). 리셋 후보는 PLC가 트립 중이고 경보가 그 트립일 때만 원인별 리셋 절차로 나옵니다. **규정 적합성**은 후보마다 예측 값 · 스킬 종류 · 명령 · PLC 모드와 상태 · 팬 운전 이력 · 예비 펌프 · 공급사 승인을 규칙에 대해 **제외 · 감점 · 경고**를 정하고 출처를 인용합니다. 제외: 예측이 인터록 한계에 닿음(설비안전규정), 원격 자동이 아닌데 원격 제어(매뉴얼), 트립 중인데 리셋이 아님, 누설 의심 시 압력 상향, 정비 중인 예비 펌프로 전환, 미승인 공급사 부품(구매규정). 감점: 팬을 오래 최대로 돌린 경우 팬 쓰는 후보. 경고: 팬만 조절하는 조치는 예측 온도가 경보 해제선에 못 미칠 수 있음, 그리고 예측 압력이 낮음. 규칙이 묻는 사실을 모르면 경고는 그대로 붙이고, 제외 · 감점 규칙은 제외로 칩니다(예: 팬 운전 이력이 없으면 팬 카드가 빠짐). 쿨러 경보를 처리하는 중에 트립이 나면 냉각 카드는 「트립 중에는 리셋만」 규칙에 모두 제외됩니다.

우선순위는 지식 지도에 적힌 점수식 하나로 매깁니다. 재료는 BSC 득실(스킬이 움직이는 변수에서 영향 관계를 따라 닿는 성과 지표의 득실, 조건부 관계는 조건이 판정되지 않으면 절반만 반영 — 지금 지식 지도에는 조건 판정 정책이 없음), 예측 온도 여유, 경고와 감점, 선례(같은 고장에서 사람이 과거에 고른 비율), 납기 긴급도와 생산 영향, 출하 품질 위험입니다. 제외된 후보는 뒤로 가고, 동점이면 낮은 등급이 승인할 수 있는 쪽이 앞에 옵니다. 점수식은 검토를 거쳐야만 바뀝니다(23장).

데이터: 판단(조치 카드 묶음). 순위가 매겨진 카드 전부(제외된 것도 이유와 함께)와 권고 한 장이 마지막 검사를 받습니다. 카드마다 그것을 고른 규칙이 인용되었는가, 권고 카드가 규정에 걸려 있지 않은가, 권고 카드에 승인 역할과 SOP 단계가 있는가. 통과하면 같은 사건에 판단으로 제출됩니다. 판단 엔진의 쓰기는 이 제출 하나뿐입니다.

판단 엔진 안의 흐름. 판단 엔진의 실행도 단계로 기록됩니다. 지식(판단 · 결정표 · 규칙과 그 검사 · 출력 · 적용 대상 · 근거를 읽음) → 사실(입력 데이터마다 출처를 보고 값을 모으고, 어디서 어떤 방법으로 읽었는지를 함께 남김) → 예측(설비별 예측 모델로 후보마다) → 후보(규칙마다 발동했는지) → 규정(후보마다 제외 · 감점 · 경고와 걸린 규칙) → 득실(예측과 BSC 경로) → 순위(권고와 순서) → 카드 검사 → 제출. 사실을 모을 때 판단 엔진은 조사 도구와 같은 읽기 통로(지식 지도 템플릿 · 기록 DB · 업무 진행의 설비 상태 · 회사 시스템 · 공급사 노드 · 선례)를 씁니다 (96). 카드 검사는 가드레일의 카드 규칙을 씁니다 (97). 같은 엔진이 세 군데서 쓰입니다. 경보 처리의 본 판단, 승인 순간의 재평가와 변경안 미리보기(14장), 그리고 포털의 가정 실험입니다 (98). AI 직원 쪽의 판단 엔진 서버(21장)도 이 엔진을 도구로 감싼 것이라, 누가 부르든 같은 지식과 같은 사실이면 같은 카드와 순위가 나옵니다.

14장 사람이 결정한다

담당자는 같은 PC의 브라우저로 포털(nginx로 내놓은 정적 화면)을 엽니다 (50). 포털에는 서버 쪽 중계가 없고, 브라우저가 각 서비스에 직접 요청을 보냅니다(HTTP) (51). 「이상 확인 · 조치」 화면의 카드마다 원인과 근거, 예상 온도, 점수가 어떻게 나왔는지, 규정 경고 · 감점, 선례, 그리고 **승인할 역할**이 적혀 있습니다. 즉시 완화 중 가벼운 것은 운전원이, 생산을 줄이는 것은 생산관리자가, 작업지시는 설비보전팀장이 승인하는 식입니다. 카드의 승인 역할과 같은 역할이거나 더 높은 등급이어야 하고(같은 등급의 다른 역할은 안 됨), 승인 역할이 없는 카드는 누구나 승인합니다. 규정에 걸린 카드, 카드 없이 원자 명령만 승인하는 것은 막혀 있습니다. 권고와 다른 카드를 고르면 「권고와 다른 안 승인」으로 기록됩니다. 이 이야기에서는 「팬 최대 + 부하 조금 줄임」이 골라집니다.

승인자의 이름과 역할은 화면에서 스스로 적습니다. 로그인이나 인증이 없어서 역할을 바꿔 적어 권한 검사를 연습해 볼 수 있습니다(알려진 한계). 고른 카드의 팬 · 부하 값을 SOP 범위 안에서 바꾸면, 새 예측을 계산한 미리보기를 보고 확인한 뒤 그 내용이 지울 수 없는 **검토본**(지문이 붙은 사본)으로 남고, 승인은 그 검토본 그대로 이루어집니다.

데이터: 승인. 업무 진행(process)은 고른 것을 그대로 믿지 않고, 승인 순간 조사관에게 지금의 사실로 다시 평가해 달라고 합니다(HTTP) (59). 재평가와 변경안 미리보기는 조사관의 판단 엔진이 하고, 조사관이 응답하지 않으면 승인하지 않습니다. 다시 보니 후보가 아니거나, 필요한 사실을 모르거나, 카드 검사에 걸리거나, 예측이 허용폭보다 나빠졌거나, 카드의 주요 항목 · 역할 · 업무 입력 · 순위 입력 · 출처가 바뀌었으면 거절합니다. 기본 방식에서는 거절되어도 사건은 승인 대기기에 그대로 있고 자동으로 다시 판단하지 않습니다. 통과해야 명령과 작업지시 요청이 정해집니다. 반려는 「업무 프로세스」 화면에서 판단을 반려할 때(사유 필수)만 있고, 사건은 「운전원 반려」로 끝납니다. AI가 설비에 직접 명령하는 길은 그림 어디에도 없습니다.

재평가는 무엇을 보나. 재평가는 지금의 지식 지도와 원천에서 사실을 다시 읽어, 사람이 고른 카드가 지금도 같은 카드인지 확인합니다. 새 판단을 제출하지도, 무엇을 실행하지도 않습니다. 적용되는 규칙의 사실을 모르면 그것을 「규정에 맞다」는 증거로 치지 않고, 내용이 실질적으로 바뀌었으면 새 판단과 새 승인을 요구합니다. 변경안 미리보기도 같은 길입니다. 사람이 팬 · 부하 값을 바꾸면 업무 진행이 조사관에게 새 예측과 판정을 받아 검토본으로 만들고, 업무 진행의 SQLite에 먼저 저장한 뒤에야 화면에 돌려줍니다. 승인은 그 검토본을 그대로 복사해 씁니다. 검토본을 만들거나 열어 보는 것만으로는 아무것도 승인되지 않습니다.

가정 실험. 포털의 「조치 판단 규칙」 화면은 같은 판단 엔진을 제출 없이 돌립니다 (79). 설비와 패턴을 고르고 운전 모드 · PLC 상태 · 팬 최대 운전 이력 · 예비 펌프 같은 사실을 바꿔 넣으면, 원인 판정 결정표 · 카드 · 순위 · 규칙별 발동 여부 · 입력별 출처와 값과 읽은 방법이 나옵니다. 가정한 사실이 든 판단은 제출할 수 없게 막혀 있어, 실험이 실제 사건을 건드리지 않습니다. 「운전 모드를 원격 수동으로 바꾸면 제어 카드가 모두 빠진다」 「예비 펌프가 정비 중이면 전환 카드가 빠진다」 같은 규칙의 반응을 사람이 직접 확인하는 화면입니다.

15장 다시 설비로: 검사 두 겹

데이터: 명령. 업무 진행이 명령을 장부의 action.cmd 칸에 적습니다(Kafka) (36). 명령에는 명령 번호 · 설비 · 사건 · 출처(HITL) · 조치 목록(원자 조치 이름과 값) · 승인자 · 만든 시각 · 짧은 만료 시각이 실립니다. 같은 일을 감사 칸에도 남깁니다. 「명령 보냄」을 먼저 저장한 뒤 보내고, 명령을 다시 보내지는 않습니다.

검사 1 — 명령 게이트웨이. 게이트웨이가 명령을 가장 최근 것부터 읽어 (19) 차례로 검사합니다. 서식(필요한 칸이 다 있고 출처가 HITL인가) → 허용된 조치인가(팬 · 부하 · 리셋 · 정지 · 펌프 선택과 옛 카드용 이름 둘. 압력 설정은 지식 지도에 있어도 여기서 막힘) → 만료되지 않았나 → 최근 명령과 번호가 겹치지 않나 → 설비가 원격 자동 모드인가(설비 상태를 한 번도 못 봤으면 거절)와 전체 초당 건수 제한. 모드를 알기 위해 게이트웨이는 게시판의 설비 상태 칸을 늘 구독합니다(MQTT) (22). 처음 걸린 검사에서 멈춥니다. 통과하면 조치를 PLC가 아는 이름으로 바꿉니다(팬 → 팬 설정값, 부하 → 부하 설정값, 펌프 A/B → 숫자, 리셋 · 정지 → 켜짐). 그리고 게시판의 자동 명령 칸에 남기지 않는 쪽지로 붙입니다(MQTT, QoS 1) (20). 통과든 거부든 검사 이름과 이유를 감사 칸에 남깁니다(Kafka) (23). 게이트웨이의 기억(최근 명령 번호 · 초당 건수 · 설비 상태 · 최근 판정 목록)은 메모리에만 있습니다.

검사 2 — PLC. 옛지가 자동 명령 칸에서 명령을 받아 (7) PLC로 넘깁니다 (8). PLC는 2장의 검사를 다시 합니다(모드 · PLC 시계로 만료 · 중복 · 범위 · 트립). 전부 통과해야 팬과 부하를 바꿉니다 (2). 두 겹으로 보는 이유는 네트워크 위의 검사와 설비 옆의 검사가 서로를 믿지 않게 하려는 것입니다.

16장 돌아오는 대답, 회복, 종결

데이터: ACK(명령 결과). PLC의 결과(완료 또는 거절과 이유)는 설비 상태 쪽지에 실려 게시판에 붙고(MQTT, retain) (6), 수집 다리가 구독해 (9) 장부의 plant.status 칸으로 올립니다 (10). 업무 진행은 이 칸과 경보 칸을 읽어 (37) 명령 번호가 같은 결과를 기다립니다. 정해진 시간 안에 오지 않으면 「결과 시간 초과」로, 거절이 오면 즉시 사람에게 넘깁니다. 저장 심부름꾼은 같은 결과를 기록 DB의 조치 줄에 붙입니다.

기본 방식의 사건은 상태기계(미니 BPMN)입니다. 단계와 넘어가는 이유는 이렇습니다. **카드 접수**(조사관이 안내 카드를 제출해 사건이 생김) → **승인 대기**(사람을 기다림, 판단 카드가 붙음) → **명령 발행**(승인과 재평가가 통과해 명령을 장부에 적음) → **결과 대기**(같은 명령 번호의 결과를 기다림) → **결과 받음**(PLC가 완료라고 답함) → **재관측**(정해진 설비 시간을 기다렸다가 판정) → **회복**(경보 해제와 패턴별 회복 기준을 함께 확인) → **작업지시 생성**(CMMS가 작업지시 번호를 돌려줌) → **종결**. 실패 쪽 끝은 셋입니다. **사람에게 넘김**(ESCALATED)에는 사유가 함께 남습니다. 결과 시간 초과, PLC 거절, 조치 실패(재관측에서 회복 못 함), 이 흐름이 지원하지 않는 경보 패턴, 회복 기준이 정의되지 않은 패턴, 업무 진행 재시작 검토. **운전원 반려**는 판단을 사유와 함께 반려했을 때, **조치 없이 해제**는 승인 전에 경보가 먼저 풀렸을 때입니다. 작업지시만 있는 카드는 명령 · 결과 · 재관측 단계를 건너뛰고 곧바로 작업지시로 갑니다. 명령은 한 번만 보내고 다시 보내지 않습니다. 결과 대기와 재관측의 타이머는 업무 진행의 메모리에 있고, 사건은 단계가 바뀔 때마다 SQLite 파일에 저장됩니다. 판단(조치 카드 묶음)의 상태는 따로 움직입니다. 승인 대기 → 승인됨 → 실행됨 또는 일부만 실행, 아니면 반려.

재관측. 업무 진행은 명령이 들어갔다고 끝내지 않습니다. 결과를 받은 뒤 일정 설비 시간을 기다렸다가 한 번, 기록 DB에서 「경보 해제 + 패턴별 회복 기준(쿨러는 TS1이 기준 아래)」을 확인합니다(SQL) (38). 값은 이미 기준 안인데 해제가 아직 안 왔으면 몇 번 짧게 더 기다리고, 끝내 맞지 않으면 「조치 실패」로 사람에게 넘깁니다. 이번에는 팬과 부하가 바뀌자 기름이 식고, 검사관이 해제식이 유지된 것을 보고 같은 경보 번호로 「풀림」을 장부에 적습니다 (13).

데이터: 작업지시와 회사 거래. 회복이 확인되면 업무 진행이 회사 시스템 목업(ERP · MES · CMMS · QMS · SCM · EMS)의 CMMS에 정비 작업지시를 냅니다(HTTP) (39). 무엇을 낼지는 승인할 때 이미 정해 둡니다. 고른 카드에 작업지시가 있으면 그것을, 이 이야기처럼 즉시 완하만 골랐으면 「선택한 조치 이후 정비 점검」을 냅니다. 팬과 부하로 버티는 것은 시간을 버는 것이지 고친 것이 아니기 때문입니다. CMMS가 작업지시 번호를 돌려줘야 사건을 닫고, 실패하면 「회복」에 머물며 사람이 다시 보낼 수 있습니다. 작업지시만 있는 카드는 재관측 없이 승인 즉시 작업지시를 내고 닫고, 부품 구매요청이 든 카드는 승인 즉시 ERP에 구매요청을 냅니다. 회사 시스템에 실제로 실행되는 거래는 작업지시와 구매요청 둘뿐이고, 그 밖의 거래가 든 승인은 「일부만 실행」으로 남습니다. 회사 시스템은 같은 요청 열쇠로 같은 일을 두 번 시키면 처음 결과를 돌려주고, 내용이 다르면 거부하며, 거래마다 원장에 전후 상태를 남깁니다.

회사 시스템의 데이터가 어디 있는지는 설정으로 갑니다. 기본(memory)이면 읽는 데이터는 프로그램에 고정된 표이고 자기 파일(SQLite)에는 실행 상태와 원장만 둡니다. 지금 설정(supabase)이면 업무 기록부 Supabase의 기업 데이터(ent 스키마)가 원천입니다(SQL) (49). AI 직원이 읽는 업무 데이터도 ent이므로, 둘이 같은 데이터를 보려면 이 설정이어야 합니다.

데이터: 사례와 선례. 업무 진행은 사건이 단계를 옮길 때마다 사건을, 승인하는 순간 판단 사례(고른 카드 · 권고를 따랐는지 · 사유)를 지식 지도의 운영 기록 층으로 옮깁니다(Bolt) (40). 반려된 판단은 사례를 남기지 않고, 결과가 나뉘었던 선택도 사례로 남습니다. 다음에 같은 고장이 오면 13장의 판단 엔진이 이 사례를 선례로 읽습니다. 사건 저장소(SQLite)가 원본이고 지식 지도는 비취 둔 사본이라, 옮기기는 대기열을 거쳐 순서를 지키고 실패하면 다시 합니다.

17장 회사 시스템: 여섯 시스템 · 거래 · 보상 · 원장

회사 시스템 목업(enterprise-sim)은 한 프로그램 안에서 여섯 시스템을 흉내 냅니다. 회사 · 고객 · 금액은 모두 교육용 가상 데이터입니다. 읽기는 시스템마다 「시스템 이름 · 판단 엔진이 쓰는 사실 · 사람이 화면에서 볼 기록」의 같은 모양으로 돌려줍니다. **MES**는 설비마다 지금 돌리는 생산오더(품목 · 고객 · 납기 · 남은 수량 · 시간당 생산량)와 옮겨 갈 수 있는 대체 설비 · 전환 시간을, **ERP**는 그 오더의 판매 계약(고객 등급 · 지연 보상 · 고장 비용 · 클레임 비용)과 완제품 재고 · 출하 예정을, **CMMS**는 쿨러 보전 프로필(세척 시간과 비용 · 다음 야간 정비창 · 예비 펌프 가용)과 세척 이력을, **QMS**는 최근 고온 구간에 만든 출하 대기 로트(고객 · 수량 · 불량 가능성 · 클레임 규모)를, **SCM**은 부품별 공급사 견적(가격 · 불량률 · 납기 · 승인 여부)을, **EMS**는 계약 전력 · 현재 수요 · 피크 시간대 · 외기 온도를 내놓습니다. 납기나 출하까지 남은 시간은 저장된 시각에서 매번 다시 계산하고, 시나리오를 다시 시작하면 시각의 기준점을 옮겨 같은 장면이 재현됩니다.

판단 엔진은 이 숫자들을 지식 지도의 입력 데이터 출처를 따라 읽습니다. 긴급 오더 남은 시간은 MES, 지연 보상과 고객 등급은 ERP, 예비 펌프 가용은 CMMS, 출하 대기 로트는 QMS입니다 (31). 공급사 승인은 SCM이 원천이지만 판단 엔진은 지식 지도의 공급사 노드에서 읽고, 그 노드는 주기 맞추기가 SCM에서 옮겨 둡니다(23장). EMS의 전력 피크나 MES의 대체

설비 같은 숫자는 화면과 AI 직원이 볼 수 있게 들어 있지만, 기본 지식에서 판단 규칙의 입력으로 연결된 것은 위의 것들입니다. 다른 숫자를 판단에 쓰려면 DDL 등록으로 입력 데이터를 만들고 순위 정책에 이름을 더하면 됩니다.

업무 기록부 Supabase의 ent 스키마가 같은 내용을 표로 가집니다. 읽는 표(설비 · 생산오더 · 판매 계약 · 완제품 재고 · 보전 프로필 · 보전 이력 · 품질 프로필 · 부품 · 공급사 · 에너지 수요)와 거래 표(작업지시 · 구매요청 · 출하 · 로트 처분 · EMS 조치 · 원장)이고, 표를 읽는 함수들이 HTTP 읽기와 같은 모양을 돌려줍니다. supabase 설정에서는 회사 시스템 · 판단 엔진의 물리 입력 · 회사 데이터 서버가 모두 이 표를 보므로, 「AI가 본 숫자」와 「업무 진행이 바꾼 숫자」가 같은 행이 됩니다.

실행 쪽. 쓰기는 업무 진행만 합니다. 조사관도 AI 직원도 회사 시스템에 쓰지 못합니다. 실행할 수 있는 거래는 시스템마다 정해져 있습니다. CMMS 정비 일정(작업지시), MES 생산 재배정, ERP 부품 구매요청, QMS 로트 격리와 로트 출하 승인, ERP 대체 출하, EMS 수요 제어. 이 가운데 지금 판단 카드가 실제로 쓰는 것은 작업지시와 구매요청이고, 다른 거래가 든 승인은 「일부만 실행」으로 남습니다. 거래는 「판단 · 고른 안 · 스킬 · 설비 · 값」을 한 요청으로 받고, 같은 요청 열쇠로 다시 오면 처음 결과를 돌려주며 내용이 다르면 거부합니다. 구매요청은 비승인 공급사를 거부하고, 재배정은 대체 설비가 맞는지 봅니다.

보상과 되돌릴 수 없는 것. 되돌릴 수 있는 거래에는 정확한 역거래가 있습니다. 작업지시 → 작업지시 취소(CMMS), 구매요청 → 구매요청 취소(ERP), 생산 재배정 → 재배정 원복(MES), 로트 격리 → 격리 해제(QMS). 역거래는 원래 거래의 참조 번호를 들고 오고, 그 기록이 원래 거래가 만든 상태 그대로일 때만 됩니다. 이미 다음 단계로 넘어갔으면 거부합니다. 로트 출하 승인 · 대체 출하 · 수요 제어는 되돌릴 수 없습니다(출하 절차로 넘어갔거나, 물류가 움직였거나, 외부에 이미 전달됨). 이것들은 되돌리는 척하지 않고, 업무 진행이 사람의 확인을 받습니다(20장).

원장. 거래와 보상은 모두 원장에 한 줄씩 남습니다. 어느 시스템, 어느 판단의 어느 스킬, 어떤 참조 번호가 생겼는지, 그리고 **바뀌기 전과 후의 상태**입니다. 보상 줄은 자신이 되돌린 원래 거래를 가리킵니다. 그래서 「이 작업지시는 이 판단에서 생겼고, 이 재작업에서 취소되었다」가 원장만으로 이어집니다. supabase 설정이면 거래는 업무 기록부의 ent 표에서 잠금을 걸고 실행되어, 같은 거래가 동시에 두 번 들어가지 않습니다 (95). 업무 진행은 판단별로 원장을 조회해 효과를 확인하고, 포털 「업무 프로세스」 화면의 회사 시스템 보드는 시스템별 상태와 거래 이력을 직접 읽어 보여 줍니다 (82). memory 설정이면 원장과 실행 상태는 회사 시스템 자기 파일(SQLite)에 남습니다.

18장 다른 두 고장과 트립

펌프 내부 누설. 「펌프 A 누설」을 넣으면 토출 압력과 유량이 떨어집니다. 탐지 정의는 「운전 중 + 압력 낮음 + 유량 낮음 + 부하가 충분히 높음」입니다. 부하가 낮으면 압력이 원래 낮으므로 부하 조건이 들어갔고, 그래서 부하를 줄이면 경보는 풀리지만 누설은 가려진 것일 뿐입니다. 정지나 트립 중에는 울리지도 풀리지도 않습니다. 원인은 축 씰 마모, 증거는 압력과 유량의 평균입니다. 후보는 예비 펌프 전환(B는 새지 않아 압력이 돌아옴), 부하 줄이기, 축 씰 교체(부품 구매요청 + 작업지시), 그리고 규정에서 제외되는 옛 절차 압력 상향입니다. 회복 기준은 압력입니다.

팬 진동 상승. 「팬 베어링 마모」를 넣으면 진동이 오릅니다. 탐지는 「진동 높음 + 오르는 중」, 해제는 「진동이 해제선 아래」입니다. 원인은 베어링 마모, 증거는 진동이 최근 구간에서 크게 움직였는가와 과열이 아닌가입니다. 진동은 팬이 빠를수록 커지므로, 팬을 늦추면 진동은 줄지만 쿨러 힘이 줄어 기름이 데워지고, 팬을 최대로 올리면 진동 인터록에 걸려

트립합니다. 쿨러 고장과 반대 방향의 맞바꿈이라 판단 엔진의 특실이 갑니다. 근본 조치는 계획 정지와 베어링 교체 (공장장 승인), 베어링 교체 작업지시입니다. 회복 기준은 진동입니다.

트립과 오일 분석. 트립 경보는 「설비 이상 조치」 흐름이 받지 않는 패턴입니다. 조사관은 보류하고, 사건은 곧바로 사람에게 넘겨지며, 인스턴스 방식이면 「미지원 경보 현장 검토」 프로세스로 갑니다(19장). 오일 분석 이탈은 사람이 결과를 넣는 패턴으로 지식에만 있고, 실제로 탐지하는 코드는 없습니다.

펌프와 팬 두 고장도 쿨러와 똑같이 장부 → 검사관 → 조사관 → 판단 엔진 → 사람 → 게이트웨이 → PLC의 길을 지냅니다. 강의용 「결함 복구」는 숨은 값을 천천히 정상으로 되돌립니다.

19장 BPMN 정의서로 본 같은 흐름: 인스턴스

인스턴스 방식에서는 일의 순서를 코드가 아니라 BPMN 정의서 「설비 이상 조치」가 정하고, 정의서는 8장 프로세스 층의 흐름 노드와 하나하나 대응합니다. 정의서의 모양은 ProcessGPT와 같습니다(AI 몫은 사용자 작업 + 에이전트 완료 방식 + cliagents 실행). 역할은 AI 에이전트 · 운전원 · 생산관리자 · SCADA(PLC 명령) · 프로세스(타이머 · 재관측) · CMMS(작업지시)입니다. 이 방식에서는 조사관의 「경보마다 자동 조사」가 꺼지고 경보는 업무 진행이 직접 받습니다. 단, 설비로 내려가는 명령 · 결과 대기 · 재관측은 기본 방식의 사건을 함께 열어 15~16장과 똑같이 처리합니다. 인스턴스를 열 때 업무 진행은 PLC 경로를 맡을 사건을 같은 경보 번호 · 설비 · 패턴으로 함께 만들고, 「PLC 명령 발행」 · 「재관측」 서비스 작업을 그 사건에 맡깁니다 (84). 사건의 단계가 바뀌면 그 변화가 서비스 작업의 완료와 「회복?」 게이트웨이가 읽을 값으로 인스턴스에 돌아옵니다 (85). 사건 번호는 AI가 만드는 값이 아니라 업무 진행이 시작할 때 넣는 값입니다.

데이터: 접수 기록. 업무 진행은 장부의 경보와 설비 상태를 먼저 업무 기록부 Supabase(PostgreSQL)의 **접수함**에 원문 그대로 저장한 뒤에야 「다 읽었다」고 장부에 알립니다 (66). 그래서 꺼졌다 켜져도 받은 경보를 잃지 않습니다. 접수 행과 설비별 최신 상태는 업무 기록부의 버팀 장치 표에 남고 (90), 엔진이 설비별 임대를 걸어 접수 행을 처리하면서 인스턴스를 시작하거나 해제 · 결과를 해당 사건에 반영합니다 (83). 접수 순간 경보 정책도 함께 고정됩니다. 정책상 이 흐름으로 받는 패턴은 쿨러 · 펌프 누설 · 팬 진동이고(각각 회복 기준이 불음), 트립 같은 나머지는 별도 정의 「미지원 경보 현장 검토」로 보내고 사건은 즉시 사람에게 넘깁니다. 그 정의는 생산관리자 작업 하나뿐이고, 경보 원문과 트립 사유를 확인해 메모를 남기면 끝나며, 「검토 완료는 회복이나 제어 승인이 아니다」라고 적혀 있습니다.

데이터: 인스턴스 · 할 일 · 이벤트. ① 시작 이벤트 「경보 수신」(메시지 이벤트): 경보 하나가 인스턴스 하나를 엽니다. 정의서에 상관 키는 설비로 적혀 있지만, 실제로 같은 경보로 두 번 열지 않게 막는 것은 경보 번호의 유일 색인입니다. 엔진은 인스턴스와 할 일(workitem)을 업무 기록부에 만들고(SQL) (41) 처음부터 모든 작업을 「예정」으로 펼쳐 둡니다. ② AI 에이전트의 사용자 작업 넷 — 원인 진단 · 조치 후보 조회 · 규정 검토 · 우선순위와 카드 작성 — 이 차례로 열리고, 11장의 네 판단(12 · 13장의 계산)에 대응합니다. 정의서가 작업에 넘기는 입력은 후보와 설비뿐이고, 무엇을 더 볼지는 작업 지시문에 적혀 있습니다. ③ 사용자 작업 「조치 카드 선택(HITL)」: 운전원이 카드를 고릅니다. **거부 버튼은 없습니다.** 이 작업에는 **경계 타이머 이벤트**가 붙어 있어, 정해진 설비 시간 안에 고르지 않으면 선택을 더 받지 않고 「상급자 호출」 작업으로 넘어가 생산관리자가 맡습니다. 타이머는 승인이 접수되는 순간 멈춥니다. ④ 배타 게이트웨이 「즉시 제어 포함?」: 고른 카드에 PLC 명령이 있으면 「PLC 명령 발행」(서비스 작업, 15장의 길)으로, 없으면 바로 「정비 작업지시」로. ⑤ 「재관측」 서비스 작업 뒤 배타 게이트웨이 「회복?」: 함께 열린 사건이 회복 · 작업지시 생성 ·

종결이면 회복, 사람에게 넘김이면 미회복입니다. 회복이면 「정비 작업지시」(CMMS) → 종료 이벤트 「종결」, 아니면 「상급자 호출」 → 종료 이벤트 「에스컬레이션 종료」. 명령을 내기 전에 사건이 먼저 끝나면 열린 작업을 취소하고 상급자 호출로 갑니다.

게이트웨이 조건은 자연어가 아니라 허락된 작은 식으로만 쓰고, 참이 하나면 그 길, 여럿이면 우선순위, 없으면 기본 길로 가며, 판정을 작업 기록에 남깁니다. 가지 않은 길의 작업은 「취소」, 서비스 작업에 닿으면 「제출됨」이 됩니다. 서비스 작업이 여러 번 실패하면 「대기」로 두고 오류를 남겨 사람이 보게 합니다. 엔진은 짧은 주기로 승인 다시 전달 → 제출된 작업 처리 → 서비스 작업 재확인 → 끝난 사건 정리 → 지식 지도 투영 재시도 → 타이머 확인을 돌고, 오래 묵은 처리를 가끔 풀어 줍니다.

정의서 안의 데이터 · 역할 · 품. 정의서에는 인스턴스 변수가 선언되어 있습니다. 설비 · 원천 경보 · 패턴 · 판정 원인 · 고장 유형 · 안내 카드 · 후보 목록 · 규정 판정 · 판단(카드 묶음)과 그 번호 · 사건 번호 · 고른 스킬과 그 종류 · 회복 여부 · 작업지시 결과 · 상급자 메모입니다. 작업마다 읽는 변수와 만드는 변수가 정해져 있습니다. 원인 진단은 패턴 · 설비 · 경보를 읽어 원인 · 고장 유형 · 안내 카드를, 조치 후보 조치는 고장 유형 · 원인을 읽어 후보 목록을, 규정 검토는 후보를 읽어 규정 판정을, 우선순위 · 카드 작성은 후보 · 규정 판정 · 사건을 읽어 판단과 판단 번호를, 카드 선택은 판단을 읽어 고른 스킬과 그 종류를 만듭니다. PLC 명령 발행은 고른 스킬과 사건을 읽고, 재관측은 사건을 읽어 회복 여부를, 정비 작업지시는 고른 스킬 · 설비를 읽어 작업지시 결과를, 상급자 호출은 판단 · 사건 · 회복 여부를 읽어 메모를 만듭니다. 첫 게이트웨이는 「고른 스킬 종류」를, 둘째는 「회복 여부」를 조건 데이터로 씁니다. 사람과 AI가 내는 결과는 작업마다 정해진 **품**(필드 목록)으로 받고, 품은 정의서 판본 안에 함께 고정됩니다. AI 작업에는 지켜야 할 확인점(예: 모든 주장에 온톨로지 노드 인용, 데이터가 묵었으면 보류)과 지시문이 붙어 있습니다. 정의서의 역할 목록(AI 에이전트 · 운전원 · 생산관리자 · SCADA · 프로세스 · CMMS)은 온톨로지의 역할 · 시스템으로 풀립니다. 정의서에는 경보 정책도 들어 있습니다. 이 흐름이 받는 패턴과 회복 기준(어느 값을 어느 방향으로 보고 해제가 꼭 필요한지), 그리고 받지 않는 경보를 보낼 정의(미지원 경보 현장 검토)입니다.

AI 몫을 누가 하나. 설정 「AI 다리(AGENT_BRIDGE)」가 정합니다. legacy(기본값, 지금 설정)이면 인스턴스 엔진이 12 · 13장의 조사관에게 경보를 보내 계산을 맡기고, 그 결과로 AI 작업 넷을 한꺼번에 채웁니다(HTTP, 요청은 임대로 맡겨 실패해도 다시 시도) (42). 이 다리는 할 일 행에 임대를 걸고, 조사관이 돌려준 결과 묶음을 먼저 행에 보관한 뒤에야 AI 작업들에 게시합니다 (94). 중간에 끊기면 임대가 끝난 뒤 다시 시도하고, 늦게 도착한 옛 응답은 게시되지 않습니다. off이면 AI 직원(21장)이 말합니다. 둘을 동시에 켜면 같은 작업을 서로 집으려 다텍니다. 승인을 실제로 전달하기 직전과 명령을 내기 직전에도 업무 진행은 조사관에게 재평가를 받고, 사람이 값을 바꾼 검토본도 조사관이 계산합니다(HTTP) (60).

실행 중인 인스턴스가 업무 기록부에 남기는 모양. 인스턴스 하나는 인스턴스 행 하나입니다(상태 실행 중 → 완료 · 취소, 정의와 판본, 역할 묶음, 변수 목록, 지금 활성인 작업, 참여자, 재작업 세대). 작업마다 할 일 행이 하나씩 생깁니다(작업 이름 · 역할 · 맡은 사람이나 에이전트 · 상태 · 에이전트 방식 · 실행 상태 · 결과 · 세대 · 임대 · 기한). 경계 타이머도 기한이 붙은 할 일 행이고, 기한이 지나면 엔진이 그 행을 터뜨려 붙어 있던 작업을 취소하고 상급자 호출로 갑니다. 이벤트 행에는 정해진 사건만 시간순으로 쌓입니다. 원천 접수함 행과 설비별 최신 상태 행이 경보와 상태를, 승인 전달 행이 사람의 선택을, 재작업 영수증과 효과 영수증이 되돌리기를 기록합니다(20장). 이 행들이 바뀔 때마다 데이터베이스가 실행 투영 대기열에 한 줄을 넣고, 업무 진행이 그 대기열을 읽어 지식 지도로 옮깁니다 (93).

인스턴스 · 정의서 · 할 일은 지식 지도에도 「실행 층」으로 비춰 둡니다(판본 · 흐름 노드 · 순서 흐름 · 인스턴스 · 작업과, 어떤 정의의 어느 판을 썼는지 · 어느 역할에 묶였는지 · 어느 설비의 어느 사건을 다루는지 · 어느 흐름 노드를 실행했는지)(Bolt) (67). 관계 이름으로 보면, 정의 판본 노드가 판본별 흐름 노드와 순서 흐름을 가지고 판본의 흐름 노드는 프로세스 층의 같은 의미 노드를 가리킵니다(MAPS_TO). 프로세스 인스턴스 노드는 판본을 쓰고(USES_VERSION) 지식 지도의 프로세스를 가리키며(INSTANCE_OF) 역할에 묶이고(ROLE_BOUND) 설비와 사건을 가리킵니다(ON_ASSET · HANDLES). 작업 노드는 인스턴스에 속하고(IN_INSTANCE) 실행한 흐름 노드를 가리키며(EXECUTES) 맡은 쪽에 배정됩니다(ASSIGNED_TO). 판본을 바꿔도 옛 판본의 노드가 남아, 정의를 고친 뒤에도 예전 실행과 비교할 수 있습니다. 대상이 지식 지도에 없으면 관계 대신 연결 경고를 남깁니다. 업무 기록부의 변경이 대기열을 거쳐 지식 지도로 가고, 판 번호와 지문으로 순서가 뒤바뀌는 것을 막습니다. 작업지시 · 구매요청 · 보상 거래와 원장 조회는 인스턴스 엔진이 회사 시스템에 직접 요청합니다(HTTP) (68).

20장 인스턴스가 버티는 법: 접수함 · 승인 전달 · 재작업 · 보상 · 투영

접수함. 장부의 경보와 설비 상태는 접수함에 먼저 저장된 뒤에야 장부 책갈피가 넘어갑니다. 저장에 실패하면 넘기지 않고 간격을 늘리며 다시 합니다. 접수 기록에는 장부의 위치, 원문(지문 포함), 접수 순간의 경보 정책이 남고, 상태는 대기 · 처리 중 · 처리됨 · 기다림 · 실패 · 중복 · 충돌 · 무효입니다. 설비마다 한 건씩 임대를 걸어 처리하고, 거듭 실패하면 「실패」로 두어 사람이 이유를 적고 다시 시도합니다. 짝이 될 경보가 아직 없는 해제 · 결과는 「기다림」이 됩니다. 접수함은 설비마다 최신 상태를 따로 두어, 업무 진행이 재시작해도 PLC 모드와 상태를 되살립니다.

승인 전달. 사람이 카드를 고르면(사람 작업 제출) 곧바로 흐름이 넘어가지 않습니다. 먼저 「승인 의도」(그 순간의 카드 사본)를 「전달 대기」로 저장하고 (86), 전달할 때 다시 확인합니다. 사건이 아직 승인 대기인가, 원자 조치와 범위가 맞는가, 검토본과 실행 값이 같은가, 조사관 재평가를 통과하는가. 통과하면 사건의 판단이 승인되어 명령과 작업지시 요청이 정해지고 (87), 구매요청 같은 작업지시 외 거래를 실행한 뒤 흐름을 넘깁니다. 승인 전달 행은 판단마다 하나이고 업무 기록부에 남습니다 (91). 전달 중에 업무 진행이 죽으면 전달 표시는 되돌아가므로, 바깥 거래는 같은 요청 열쇠로 다시 와도 한 번만 일어나게 되어 있습니다. 명령 직전에 한 번 더 재평가하고, 걸리면 작업을 대기로 두고 「새 판단과 승인이 필요하다」고 적습니다. 전달 상태는 대기 · 전달됨 · 실패 · 폐기이고, 다시 전달은 같은 역할만, 폐기는 아무 효과도 없었다는 것이 확인될 때만 사유와 함께 하며 그때 인스턴스는 취소됩니다.

재작업. 지나간 작업을 다시 해야 할 때(예: 조건이 바뀌어 다른 카드를 골라야 할 때) 사람이 재작업을 요청합니다. 엔진은 인스턴스가 시작할 때 고정된 정의서 판본에서 영향 범위(제어 흐름 · 입력 · 조건 변수 · 참조 · 경계 이벤트)를 계산해 미리보기를 보여 주고, 미리보기 지문이 같아야 실행합니다. 이미 시작된 서비스 작업, 승인 검토가 필요한 상태, 값을 만든 작업이 모호한 경우, 순환 같은 사유가 있으면 막습니다. 요청자는 그 사람 작업의 역할이어야 하고 사유와 확인이 필요합니다. 실행하면 세대가 하나 올라가고, 영향받은 작업 행은 취소한 뒤 새 행을 만들며 변수를 되돌립니다 (88). 재작업 요청과 결과는 영수증으로 업무 기록부에 남고 (92), 영수증은 지우지 않으며, 새 세대에는 새 판단이 꼭 필요하며, 조사관 다리는 새 세대에 끼어들지 않습니다.

데이터: 효과와 보상 영수증. 회사 시스템에 이미 일어난 일은 재작업 전에 정리해야 합니다. 되돌릴 수 있는 효과(작업지시 취소 · 구매요청 취소 · 생산 재배정 원복 · 로트 격리 해제)는 보상 거래로 되돌리고 보상 영수증(대기 → 전달됨 · 실패)을 남깁니다. 되돌릴 수 없는 효과(출하 승인 · 대체 출하 · 수요 제어)는 원래 승인 역할 이상의 사람이 「확인했다」는 지을 수 없는 영수증을 남깁니다. 설비에 이미 들어간 명령을 자동으로 거꾸로 되돌리는 명령은 없습니다. 설비 명령도 사람이 지금 설비 상태를 확인했다는 영수증을 남기고, 명령 이력은 그대로 둔 채 사건을 새 판단으로 다시 엽니다. 결과를 기다리는 중에는 확인할 수 없습니다. 회사 시스템의 원장에는 보상마다 대응 거래가 남고, 원래 상태에서 이미 바뀌었으면 보상을 거부합니다. 모든 효과가 보상되거나 확인된 뒤에만 재작업이 진행되고, 사건은 다시 승인 대기로 열립니다.

그 밖의 안전장치. 정의서를 등록할 때 구조를 검사하고, 같은 판에 다른 내용은 거부합니다(판본 불변). 인스턴스는 시작한 판본을 끝까지 쓰고, 옛 판은 자동으로 올리지 않습니다. 시작 입력에 사건 · 승인 · 판단 번호를 넣을 수 없습니다. 의존이 있는 작업은 흐름이 도착하고 같은 세대의 앞 작업이 끝나야 시작하고, 진행 중이거나 기다리는 의존이 없어야 인스턴스가 완료됩니다. 잠금은 인스턴스 → 작업 순서로 걸고, 다른 쪽이 잡은 행은 건너뛴다. 업무 기록부의 이벤트 표에는 정해진 일(작업 시작 · 완료, 오류, 승인 접수 · 전달, 재작업, 보상, 질문 · 답, 보류, 재평가 요청)만 남고, 일반적인 상태 이동은 장부의 감사 칸에 남습니다. 포털은 이벤트 표를 짧은 주기로 읽어 실시간으로 흘러 줍니다(SSE).

기본 방식의 버팀. 기본 방식의 사건 저장은 업무 진행의 SQLite 파일입니다. 업무 진행이 재시작하면 진행 중이던 사건은 「재시작 검토」 사유로 사람에게 넘기고, 승인했지만 다 실행하지 못한 판단은 「일부만 실행」으로 표시합니다. 타이머가 메모리에 있어 재시작하면 사라지기 때문입니다. 경보를 처리하다 DB가 실패하면 장부 책갈피를 넘기지 않고 간격을 늘려 다시 하며 상태 확인에 오류를 알립니다. 사건과 판단을 지식 지도로 옮기는 일은 SQLite 대기열에 넣고 판 번호로 순서를 지키며, 실패하면 다시 하고, 지식 지도의 지문을 주기적으로 비교해 어긋나면 다시 옮깁니다. 수동 복구 도구도 있습니다.

사례 투영과 지식이 바뀐 뒤의 맞추기. 기본 방식의 사건과 판단 사례는 업무 진행의 SQLite가 원본이고, 원본을 저장하는 같은 커밋에 「지식 지도로 옮길 일」을 대기열에 넣습니다 (89). 투영기는 짧은 주기로 대기열을 읽어 지식 지도에 사건 · 판단 사례 노드와 관계를 쓰고 (40), 지식 지도에 남긴 판 번호와 내용 지문이 원본과 정확히 같다는 영수증을 받아야 대기열에서 지웁니다. 늦게 도착한 옛 투영은 판 번호 울타리에 막힙니다. 잇는 대상(설비 · 패턴 · 원인 · 스킬 · 역할)이 지식 지도에 없으면 관계 대신 투영 경고를 남깁니다. 투영은 원본에 저장된 것만 옮기고, 명령이나 승인을 다시 하지 않습니다. 사람이 지식을 고치거나 매뉴얼을 적재해 노드가 바뀌면, 업무 진행은 사례 · 실행 투영이 쓰는 연결 열쇠의 지문을 주기적으로 비교해 어긋난 연결만 다시 잇습니다. 진단이나 승인을 다시 하는 일이 아닙니다. 원본과 사본을 비교해 다시 보내도록 줄을 세우는 수리 도구도 있고, 수리 기록은 업무 기록부에 남습니다. 수리는 업무 기록을 되감거나 명령을 내거나 전달을 완료로 표시하지 않습니다.

할 일 임대 정리. AI 직원이 할 일을 집으면 임대가 걸리고 일하는 동안 연장됩니다. 업무 진행은 주기적으로 만료된 임대를 찾아 다른 작업자가 다시 집을 수 있게 풀고, 같은 할 일이 거듭 만료되면 실패로 두어 사람이 담게 합니다. 만료 횟수는 다른 작업자가 회수했을 때만 늘고, 사람이 답해 다시 시작하면 처음으로 돌아갑니다. 이 정리는 업무 진행 방식과 상관없이 돌고, 처리 묶음이 오래 잡혀 있으면 엔진이 가끔 풀어 줍니다.

업무 기록부의 표 묶음. 그림의 업무 기록부 따는 세 묶음입니다. ① 프로세스 표는 ProcessGPT와 같은 모양입니다. 테넌트(등록된 MCP 서버 목록 포함), 사용자(사람과 에이전트가 한 표, 에이전트 여부로 구분), 정의와 판본(판본은 바꿀 수 없음), 품, 인스턴스, 할 일(세대 · 임대 포함), 이벤트, 알림. ② 버팀 장치 표는 이 장의 장치들이 쓰는 영수증과 대기열입니다. 원천 접수함과 설비별 최신 상태, 승인 전달, 재작업 영수증, 효과 영수증, 실행 투영 대기열, 투영 수리 기록. ③ ent 기법

데이터는 17장의 회사 업무 DB입니다. 기본 표들은 행 단위 권한을 컷지만 익명 키로 읽을 수 있고(교육용 한계), 새로 만든 버팀 장치 표는 익명 · 로그인 사용자 권한을 거둬 서비스 계정만 씁니다. 회사 데이터는 읽기 전용 계정으로만 AI 쪽에 열립니다.

21장 AI 직원과 MCP 서버

AI 직원(agent-worker)은 ProcessGPT의 cliagents 작업자 역할입니다. 선택 묶음으로 컨테이너로 켜거나, 강사 PC에서 강사가 로그인해 둔 Claude Code로 직접 돌릴 수 있습니다. 업무 기록부에서 자기 몫의 할 일을 주기적으로 찾아 하나 집고, 임대를 걸어 맡으며 일하는 동안 임대를 연장합니다. 맡은 채 만료되면 다른 작업자가 이어받을 수 있고, 여러 번 만료되면 실패로 처리되어 사람이 단습니다. 결과는 정해진 서식(폼 계약)에 맞춰 조건부로 저장합니다(SQL) (43). 서식이 틀리면 같은 대화에서 고쳐 달라고 합니다.

할 일 하나가 AI 직원을 지나는 길. ① 폴링: 업무 기록부에서 「진행 중 · 에이전트 완료 방식 · 자기 몫(cliagents)」인 할 일을 하나 집습니다. 집는 순간 임대와 작업자 이름이 할 일 행에 적히고 실행 상태가 「시작」이 됩니다. ② 맥락: 할 일 행에 없는 것을 모읍니다. 그 작업의 폼(결과 계약), 사람과 에이전트 사용자 목록, 테넌트에 등록된 MCP 서버 목록, 이전 피드백. ③ 작업 공간: 할 일마다 폴더를 만들고 규칙 문서 · 할 일 내용 · 결과 자리 · 대화 기록을 둡니다 (100). ④ 연결 설정: 테넌트 목록 가운데 선언된 서버만 이 실행의 연결 설정 파일에 적습니다. ⑤ 지시문: 누가 시켰는지, 무엇을 돌려줘야 하는지(폼의 필드), 검토자가 무엇을 지적했는지를 말로 다 적고, 길면 파일로 넘깁니다. ⑥ 실행: Claude Code를 대화 없이 실행하고, 흘러나오는 사건(작업 시작 · 도구 사용 시작과 끝 · 메시지)을 포털이 아는 이벤트로 바꿔 업무 기록부의 이벤트 표에도 남깁니다. ⑦ 판정: 마지막 응답을 폼 계약에 대어 봅니다 (101). 필드가 맞으면 결과로, 서식이 틀리면 같은 대화로 고쳐 달라고 하고, 끝내 틀리면 실패로 둡니다. 도구를 쓰지 못해 막혔으면 사람에게 묻고, 근거가 없으면 보류합니다. ⑧ 저장: 임대가 아직 자기 것일 때만 결과를 저장하고, 할 일이 「제출됨」이 되면 그다음은 인스턴스 엔진이 이어받습니다.

일은 Claude Code를 자식 프로그램으로 띄워서 합니다(HTTPS로 Claude API와 대화) (45). 작업 공간 폴더는 일정 기간 뒤 정리합니다. 지켜야 할 규칙 문서가 함께 들어가고, 허락된 도구는 Neo4j 스키마 보기 · 읽기, 회사 데이터 서버 도구, 판단 엔진 서버 도구, 파일 읽기 · 찾기뿐입니다. 연결 설정에는 선언된 MCP 서버만 넣고 끝나면 비밀 값을 지우며, 자식 프로그램에는 비밀 환경 값을 넘기지 않습니다. 작업자 자신이 할 일을 집는 데 쓰는 DB 접속 정보나 그래프 암호도, 설정을 읽은 뒤 자기 환경에서 지워 자식이 물려받지 못하게 합니다. Codex를 대신 쓸 수도 있습니다(강사 PC에서만). 같은 할 일을 다시 하면 이전 대화를 이어 씁니다.

AI 직원은 사람에게 물을 수 있습니다. 그동안 작업은 「사람 대기」가 되고, 포털에서 답하면 이어서 합니다. 질문은 메모리가 아니라 작업 공간에 적혀 있어서, 답이 다음 날 와도 물었던 그 실행과 대화로 돌아갑니다. 근거를 모르거나 지지할 수 없으면 결과 대신 「보류」를 내고 영수증을 남깁니다. 사람이 작업을 취소하면 자식 프로그램을 통째로 끕니다. 시간 제한을 넘기면 실패입니다.

AI 직원은 데이터를 허락된 MCP 서버로만 봅니다(MCP) (44). **판단 엔진 서버(Streamable HTTP):** 12 · 13장의 판단 엔진을 도구로 내놓습니다 — 원인 진단, 판단 규칙 보기, 입력 정의 보기, 사실 모으기, 시계열 구조 보기 · 질의, 시스템 감시의 지표 목록 · 계열 · 질의, 카드 평가, 조치 예측, 판단 제출, 선례, 득실. 판단 엔진 코드를 그대로 감싸므로 같은 템플릿 질의 · 같은 규칙 평가 · 같은 예측을 씁니다. 「반복되는 판단은 규칙으로, 언어 모델은 모으고 설명한다」는

ProcessGPT의 원칙대로, AI 직원은 규칙 결과를 스스로 다시 계산하지 않고 이 도구에 묻습니다. **회사 데이터 서버** (Streamable HTTP) (102): 시스템별 고정 읽기 도구(MES 오더 · ERP 계약과 재고 · CMMS 이력 · QMS 로트 · SCM 공급사 · EMS 수요), 표 정의(DDL) 보기, 카탈로그, 구문 검사를 거친 읽기 전용 SELECT 한 문장(결과 크기와 시간을 제한하고, 특권 계정으로 연결되면 시작을 거부). **Neo4j 서버**(mcp-neo4j-cypher, stdio, 읽기 전용, 스키마 · 읽기 두 도구): AI 직원 안의 Claude Code가 지식 프로그램으로 띄우고 (103), 실행할 때 인터넷에서 내려받으며, 지식 지도를 읽기 전용으로만 봅니다 (104).

MCP 서버가 읽는 곳. 지식 지도(Bolt) (46), 업무 기록부의 기업 데이터(읽기 전용 계정, SQL) (47), 그리고 판단 엔진 서버는 기록 DB의 시계열(읽기 전용 계정, SQL) (62), 시스템 감시의 지표(HTTP, PromQL) (63), 회사 시스템 목록의 업무 숫자(HTTP) (64), 수집 다리의 상태(HTTP) (65)를 읽습니다. 업무 진행에게서는 최신 설비 상태를 읽고, 쓰는 일은 판단 제출 하나뿐입니다(HTTP) (48). 업무 진행은 AI 직원이 살아 있는지, 무엇을 하는지를 물어 포털에 보여 줍니다(HTTP) (71).

업무 기록부 Supabase는 로컬 개발용으로 돌고(API · PostgreSQL · 관리 화면), 처음 채울 때 테넌트 hyd와 그 MCP 서버 목록, 사람 사용자(운전원 · 생산관리자 · 정비관리자)와 시스템 계정(AI 에이전트 · SCADA · 프로세스 · CMMS), 품 정의를 넣습니다.

22장 포털 화면들

포털은 세 묶음의 메뉴로 되어 있습니다. 실습 안내(실습 홈 · 시스템 아키텍처), 설비와 사람의 판단(결함 시뮬레이션 · 이상 확인 · 조치 · 실시간 모니터링), 지식과 업무(온톨로지 지식 지도 · 에이전트 스킬 · 조치 판단 규칙 · 업무 프로세스 · 프로세스 인스턴스)입니다. 위에는 늘 열린 사건 수 · 실시간 연결 · 서비스 응답 · 설비 시각과 배율이 보입니다. 포털은 서버 쪽 중계가 없는 정적 화면이라, 화면마다 어느 서비스를 직접 부르는지가 곧 연결 구조입니다. 그림에서는 포털을 하는 일에 따라 카드 넷(실습 · 설비 화면, 프로세스 인스턴스 화면, 이상 확인 · 업무 화면, 지식 화면)으로 나뉘고, 모두 같은 포털 컨테이너입니다.

실습 홈은 전체 흐름도, 실시간 TS1, 몇 가지 통계와 안내를 보여 줍니다. **시스템 아키텍처**는 L9부터 L1까지 층별 구성과 링크를 나열하고, 직접 만든 서비스들의 상태 확인과 제품 화면(Neo4j · FUXA · Grafana · Prometheus · Redpanda · Console · EMQX)에 주기적으로 물어 응답 표시등을 켭니다. **결함 시뮬레이션**은 설비마다 카드 한 장입니다. 유온 막대와 냉각 효율 · 냉각 능력 · 팬 · 부하 · 압력 · 유량 · 진동 · 운전 펌프 · 쿨러 성능, 패턴별 탐지 단계(검사관에게 물음), PLC 상태와 트립 원인, 최근 명령 결과(게이트웨이에게 물음), 압력 추이, 진행 중인 고장이 보입니다. 사람은 여기서 고장을 넣고 복구하고, 현장 패널처럼 모드를 바꾸고, 수동으로 팬 · 부하 · 보호 정지 해제를 쓰고(이때 원격 자동이면 원격 수동으로 바뀜), 모든 설비를 초기화하고, 배율을 바꿉니다. 이 쓰기는 4장의 강의용 입구로 바로 갑니다. **실시간 모니터링**은 Grafana 패널을 끼워 보여 주고, 설비를 고를 수 있으며 Grafana가 꺼져 있으면 안내를 띄웁니다 (70).

이상 확인 · 조치는 기본 방식의 승인 화면입니다. 유압 회로도(모터 · 펌프 · 부하 · 압력 · 유온 · 냉각 효율 · 팬)에 현재 값이 색으로 보이고, 사건 목록(카드를 없고 조사관 실행만 있는 경보도 표시)과 사건의 진행 단계 · 결과 · 인터록 · 작업지시 번호 · 작업지시 다시 보내기, 안내 카드(사전확률 · 점수 · 증거 · SOP · 매뉴얼 절), 조사관의 단계별 실행 기록, 감사 기록이 있습니다. 같은 화면에 사건이 BPMN 레인 위의 어디에 있는지가 실시간으로 그려집니다. 판단 카드마다 점수

내역(BSC · 예측 · 경고 · 감점 · 선례 · 납기 · 품질)과 예측 조건과 출처를 펼쳐 볼 수 있고, 제외된 카드는 지금 조건으로 다시 계산해 볼 수 있습니다. 팬 · 부하 값을 원자 조치의 허용 범위 안에서 조절하면 「새 예측 검토」를 거쳐야 결정 버튼이 열립니다. 결정자 이름과 역할은 화면에서 고르는 값이고 인증이 아닙니다. 이 화면은 업무 진행과 조사관의 실행 기록을 직접 부릅니다 (51). 반려 버튼은 없습니다.

온톨로지 지식 지도는 설비별로 층을 열로 펼친 그래프(가치 · 프로세스 · 리소스 · 진단 · 스킬과 규칙, 그리고 외부 · 예측 · 사례를 한 열에)를 그립니다. 층을 켜고 끄고, 패턴 하나의 경로를 강조하고, 검색하고, 노드의 속성과 관계를 봅니다. 그래프와 질의 템플릿 보기(실행하지 않고 문장만)는 조사관의 읽기 통로에서 옵니다 (78). 같은 화면에 DDL 등록(23장)과 매뉴얼 등록(24장)이 있고, 이 둘은 업무 진행의 지식 관리로 갑니다. 매뉴얼 쪽에서는 예시 매뉴얼 내려받기, 구조 파서와 AI 직원 추출 중 고르기, 개정판 올리기, 보관된 원문 목록과 원본 내려받기, 글자 인식 필요 쪽 표시, SOP별 종류 · 관계 · 영향 고르기, 「검토했습니다」 확인, 등록 이력과 되돌리기를 합니다 (81).

에이전트 스킬은 스킬 목록과 상세(단계 · 고장 유형 · 원인 · 원자 조치 · 규칙 · 지표)를 보여 주고, 이름 · 설명 · 승인 역할을 고치거나 새 스킬을 만듭니다 (80). 고칠 때 판 번호가 다르면 충돌로 거절하고, 같은 요청은 다시 확인합니다. 매뉴얼에서 온 스킬은 매뉴얼 쪽에서만 바뀌도록 잠겨 있습니다. **조치 판단 규칙**은 14장의 가정 실험 화면입니다. **업무 프로세스**는 판단 목록(경보에서 온 것과 수동, 「권고와 다른 안 승인」 표시), 카드별 승인 · 반려(사유 필수), 실행 결과(완료 · 사람 승인 경유)와 이력, 그리고 회사 시스템 보드(MES 생산오더 · CMMS 작업지시 · ERP 구매요청과 출하 · QMS 로트 처분 · EMS 수요 제어)와 최근 거래 이력을 보여 줍니다. 판단은 업무 진행에, 보드는 회사 시스템에 직접 묻습니다. 순위 점수식 편집은 화면이 아니라 API로만 합니다.

프로세스 인스턴스(인스턴스 방식에서만): 위 띠에 정의 · 실행 중 · 내 할 일 · AI 질문 · 작업자 상태 · 사용할 수 있는 CLI · 대체 경로 · 배율이 보이고, 기본 방식이면 안내만 나옵니다. 실시간 이벤트 흐름(종류별 거르기 · 멈춤 · 도구별 소요 시간), AI 질문에 답하는 창구(선택지가 있으면 고르고, 답이 승인이나 권한을 바꾸지 않는다고 적혀 있음), 사람 작업 품 제출, 카드 선택 기한, 승인 전달 상태 · 다시 전달 · 폐기, 작업지시 다시 보내기, 보류와 재평가 요청, 효과(보상 · 확인), 재작업(영향 미리보기 · 막는 사유 · 요청자 · 역할 · 사유 · 확인 · 새 세대, 이전 승인은 폐기), 작업 닫기 · 실행 취소, 시작 입력과 변수 출처, 세대와 게이트웨이 판정이 보이는 할 일 목록, 최근 이벤트, 지식 지도 실행 층의 투영 상태와 그것을 보는 질의문이 있습니다. 고급 화면으로 원천 접수함(상태별 목록 · 원문 · 정의 · 이력 · 지문 · 장부 위치 · 다시 시도)과 정의 등록(같은 판본은 바꿀 수 없음)이 있습니다. 이 화면은 업무 진행의 인스턴스 기능과 이벤트 흐름(SSE)을 직접 부릅니다(HTTP) (69).

23장 지식을 고치는 길: 스킬 · 점수식 · BSC 조건 · DDL · 공급사

데이터: 지식 편집. 지식 지도는 사람이 고칠 수 있고, 고칠 때마다 흔적이 남습니다(Bolt) (72). 스킬의 이름 · 설명 · 승인 역할을 고치거나 새 스킬을 만들면 「지식 편집」 영수증이 남고, 판 번호를 비교해 그사이 남이 고쳤으면 거절합니다. 순위 점수식도 허락된 함수와 정해진 구성 요소만 쓰는 식으로 고칠 수 있습니다. BSC 조건부 관계에 조건 판정 정책을 붙일 수도 있습니다.

편집 하나가 지식 지도에서 하는 일. 새 스킬을 만들면 한 트랜잭션 안에서 스킬 노드, 단계 노드와 근거 절 연결, 실행 시스템(제어면 SCADA, 작업지시면 CMMS), 고장 유형과의 관계(즉시 완화 · 근본 조치), 영향(AFFECTS), 승인 역할이 함께 생기고, 그 고장 유형만 따지는 조치 후보 규칙에 출력 관계가 이어집니다. 그래서 다음 판단부터 곧바로 후보가 됩니다. 연결

고장 유형을 바꾸면 옛 관계와 옛 후보 연결을 걷어 내고 새로 잇습니다. 순위 점수식을 바꿀 때는 바꾼 식을 먼저 검토 (미리보기와 지문)하고, 지문이 같을 때만 규칙 노드의 정책을 바꾸면서 같은 트랜잭션에 지식 편집 영수증(무엇을 · 누가 · 이전 지문과 새 지문)을 남깁니다. 같은 요청 번호로 다시 오면 영수증을 돌려줄 뿐 두 번 바꾸지 않습니다. BSC 조건 판정 정책도 같은 절차로, 조건부 관계 하나에 「조건이 참이면 전부 · 거짓이면 반영 안 함 · 모르면 절반」을 무엇으로 판정할지 붙입니다. 지금 지식에는 붙어 있지 않아 조건부 특성은 늘 절반만 반영됩니다. 편집은 업무 진행만 하고, 조사관 · AI 직원은 지식 지도를 읽기만 합니다.

데이터: 입력 데이터(DDL 등록). 회사 업무 DB의 표 정의(DDL)를 붙여 넣으면 SQL 구문 분석기(SQLGlot)로 미리보기를 만들고, 어느 시스템의 표인지 추정한 뒤, 사람이 확인하면 지식 지도의 「입력 데이터」로 등록하고 그 표 · 칸에 연결합니다. 이것이 13장의 「물리 입력」입니다. 등록은 묶음 단위로 되돌릴 수 있고, 규칙을 SQL로 바꾸는 것은 문장을 만들어 보여 주기만 합니다. 지식 지도에는 회사 시스템마다 시스템 노드가, 고른 칸마다 입력 데이터 노드가 생기고, 입력 데이터의 변수 이름이 규칙이 검사하는 이름이 되며, 출처 관계(SOURCED_FROM)에 표와 칸이 적힙니다. 올린 DDL 문장은 실행하지 않고 구조만 읽습니다. 적재마다 묶음 노드가 생기고 누가 어떤 노드를 썼는지 소유 기록이 남아, 다른 쓰기 주체가 고친 노드는 덮지 않고 충돌로 알립니다.

주기적 맞추기. 업무 진행은 주기적으로 업무 기록부를 봅니다(SQL) (73). 등록된 표 · 칸이 그대로 있는지(정상 · 없어짐 · 형식 바뀜 · 설명 바뀜)를 비교해 입력 데이터에 표시하고, SCM의 공급사 정보(승인 여부, 부품별 가격 · 품질 · 납기)를 지식 지도의 공급사 노드와 관계로 옮깁니다 (107). 같은 주기로 AI 직원의 묶인 임대도 정리합니다. 이 일은 업무 진행 방식과 관계없이 돕니다. 원천 상태 표시는 사람이 먼저 알아채지 않아도 업무 DB의 변화가 지식 지도에 닿게 하려는 것입니다. 칸이 없어지거나 형식이 바뀌면 그 입력을 읽는 판단이 실패하거나 뜻이 다른 값을 읽게 되므로, 상태와 지금의 형식 · 설명 · 바뀐 시각을 입력 데이터 노드에 적어 돕니다. 설명만 바뀌면 형식은 같아도 업무상 뜻이 바뀌었을 수 있다는 표시입니다. 공급사 동기화는 업무 DB가 주인입니다. 공급사는 없으면 만들고, 업무 DB에 있는 부품의 견적 관계는 맞추며 업무 DB에서 사라진 견적은 지웁니다. 업무 DB에 없는 부품의 견적(처음 채운 지식에만 있는 것)은 지우지 않고 보고만 합니다.

24장 매뉴얼이 지식이 되는 길

데이터: 매뉴얼 → SOP. 담당자가 포털에서 매뉴얼(Markdown · 텍스트 · PDF)을 올리면 원문이 판본(지문)으로 보관됩니다. 암호가 걸린 PDF나 깨진 파일은 받지 않고, 같은 파일을 다시 올리면 저장된 추출을 다시 씁니다. 읽을 때마다 원문과 쪽의 지문을 다시 확인합니다. 글자가 없는 스캔 쪽이 있으면 올리기와 보관은 되지만 「글자 인식 필요」로 표시되고, 추출과 적재는 거부됩니다.

추출 방법은 둘입니다. AI 없이 제목과 번호로 읽는 구조 파서, 그리고 AI 직원(정의 「매뉴얼 원천 추출」). AI 직원 추출을 고르면 업무 진행이 그 정의의 인스턴스를 열어 보관된 원문 판본을 시작 변수에 고정하고, 그 할 일을 AI 직원이 집습니다 (105). 이 인스턴스가 끝나면 나오는 것은 인용이 달린 제안일 뿐, 지식 지도 적재도 사람의 승인도 아닙니다. 원문이 길면 Markdown 제목과 쪽 경계로 구간을 나눕니다. AI 직원은 구간마다 절 · SOP · 단계와 쪽별 검토 · 경고, 그리고 각 항목의 쪽과 인용문만 냅니다. 인용문이 원문 어디에 있는지는 서버가 찾고(그대로 일치 → 글자 · 숫자만 비교 → 낱말 사이가 조금 끼어도 허용), 여러 곳에서 찾아지면 더 긴 인용을 요구합니다. 서버는 모든 쪽에 검토 기록이 있는가, 단계 번호가 이어지는가, SOP 번호 형식, 쪽별 인용 비율을 검사하고, 문제가 있으면 의견을 돌려주며 다시 뽑게 합니다.

구간 결과를 합칠 때 같은 절이나 SOP가 두 구간에 나오면 앞의 것을 쓰고 경고하며, 구간 밖을 인용하면 거부합니다. 미리보기는 기존 SOP 번호와 겹치는지도 보여 줍니다. 사람이 검토자 이름을 적고 「검토했습니다」를 누르며, SOP마다 즉시 완화/근본 조치, 제어/작업지시, 영향, 그리고 연결할 고장 유형을 고릅니다. 확정하면 원문 인용을 다시 검사한 뒤 지식 지도에 한 트랜잭션으로 적재합니다(Bolt) (54). 판본이 바뀌었거나 그사이 밖에서 고쳤으면 거부하고, 다른 출처가 소유한 노드는 덮지 않습니다.

중요한 연결. 새 SOP를 고장 유형에 이르면, 그 고장 유형만 따지는 조치 후보 규칙에 그 스킬이 자동으로 연결되어 다음 판단부터 후보가 됩니다(PLC 상태처럼 다른 조건도 보는 규칙은 예외). 즉 매뉴얼 → 검토 → 지식 지도 → 판단 엔진의 후보로 곧장 이어집니다. 적재는 가장 최근 묶음부터, 적재 직후 상태와 같을 때만 되돌릴 수 있고, 밀린 옛 묶음은 「대체됨」이 됩니다. 매뉴얼마다 질문을 적어 두면 AI 직원이 Neo4j 서버로 이 문서의 노드를 인용해 답하고 판정받는 「골든 질문」 시험도 있습니다(API로만).

❏ 순조롭지 않을 때: 갈림길과 실패 경로

운전원이 수동 조작 중이면. 설비가 원격 수동이라 게이트웨이가 명령을 거부하고 감사 칸에 남깁니다. 업무 진행은 결과를 받지 못해 사건을 「사람에게 넘김(결과 시간 초과)」으로 바꾸고, 명령을 다시 보내지 않습니다. 거부 사유는 감사 기록(Grafana), 게이트웨이의 최근 판정 목록, 결함 시뮬레이션 화면에서 보이지만, 사건 화면에는 「시간 초과」로만 보입니다.

PLC가 거절하면. 게이트웨이를 통과했어도 PLC가 거절하면(예: 그새 트립) 결과가 「거절」로 돌아오고, 업무 진행은 기다리지 않고 바로 사람에게 넘깁니다.

아무도 조치하지 않으면. 기름이 계속 데워져 PLC가 펌프를 세우고, 검사관이 과열 정지 경보를 냅니다. 이 경보는 이 흐름이 받지 않으므로 사람에게 넘겨지고(인스턴스 방식이면 현장 검토), 식은 뒤에만 리셋할 수 있습니다. 인스턴스 방식에서 카드를 고르지 않고 기한이 지나면 상급자 호출로 갑니다.

조치가 듣지 않으면 · 먼저 풀리면 · 모르는 경보면. 재관측에서 회복을 못 하면 「조치 실패」로 넘깁니다. 승인하기 전에 경보가 먼저 풀리면 「조치 없이 해제」로 닫습니다. 패턴이나 회복 기준을 모르는 경보는 곧바로 넘깁니다. 조사관이 근거를 못 찾거나 데이터가 낡았으면 카드를 보류하고, 판단 엔진이 실패하면 카드 없이 승인 대기로 남습니다.

명령이 늦게 도착하면. 만료가 지난 명령은 게이트웨이와 PLC가 각자의 시계로 버립니다. 옛 조치가 뒤늦게 실행되는 일을 막습니다. 계측 값은 사라지면 안 되지만, 명령은 일부러 사라지게 했습니다.

되돌려야 하면. 인스턴스 방식에서는 재작업으로 되돌립니다. 회사 거래는 보상하거나 확인 영수증을 남기고, 설비 명령은 자동으로 거꾸로 돌리지 않으며, 사건은 다시 승인 대기로 열립니다.

부품이 꺼지면. 수집 다리가 꺼진 동안의 계측은 남지 않습니다. 검사관 · 저장 심부름꾼 · 업무 진행(인스턴스 방식의 접수함)은 장부의 책갈피나 자기 저장소로 이어서 읽습니다. 지식 지도가 꺼지면 검사관은 새 후보를 세지 않습니다. 조사관이 꺼지면 승인 순간의 재평가가 안 되어 승인이 거절됩니다.

요약 데이터 리니지 요약

한 줄에 하나씩, 데이터가 태어나는 곳 → 바뀌는 모양 · 이름 → 저장되는 곳 → 읽는 쪽입니다.

센서 값: 열 모델이 계산 → DAQ 필터가 보낼 것을 고름 → 엣지가 MQTT plant/설비/tag/이름 {젠 시각 · 값 · 품질} → 수집 다리가 설비 · 이름을 붙여 Kafka plant.tag → 저장 심부름꾼이 품질을 빼고 기록 DB 초 단위 값 표(→ 분 단위 요약) → 검사관 · 조사관 증거 · 판단 엔진 · 재관측 · Grafana · MCP 서버가 읽음. 운전실 화면은 MQTT에서 바로 읽음.

설비 상태: PLC가 매번 만듦(모드 · 운전 상태 · 트립 원인 · 숨은 값 · 마지막 명령 결과) → MQTT plant/설비/status(retain) → FUXA 표시 · 게이트웨이의 모드 기억 · 수집 다리 → Kafka plant.status → 검사관 트립 감시 · 업무 진행(결과 대기, 예측 입력, 인스턴스 방식이면 접수함의 설비별 최신 상태) · 저장 심부름꾼.

파형 · 특징값: (전부 보고일 때) 엣지가 파형 묶음 → MQTT plant/설비/wave → Kafka plant.wave → 검사관이 평균 · 실효값 · 기울기로 요약. TS1마다 기울기와 이상 점수 → Kafka feat.1s → 기록 DB 특징값 표 → Grafana.

경보(울림 · 풀림): 검사관이 지식 지도의 탐지 정의(또는 PLC 트립)로 판정 → 검사관 SQLite outbox → Kafka alerts(같은 경보 번호로 울림과 풀림) → 게이트웨이가 표시용으로 바꿔 MQTT plant/설비/alert(retain) → FUXA / 기록 DB 경보 표(울림 줄을 풀림이 고침) → Grafana / 조사관(기본 방식) / 업무 진행 접수함(인스턴스 방식).

원천 접수함 행: (인스턴스 방식) Kafka의 경보 · 설비 상태 → 원문과 지문, 장부 위치, 접수 순간의 경보 정책을 붙여 Supabase 접수함에 먼저 저장 → 장부 책갈피를 넘김 → 엔진이 설비별 임대료 처리(처리됨 · 기다림 · 실패 · 중복 · 충돌 · 무효) → 인스턴스 시작, 사건의 결과 · 해제 반영 → 포털의 원천 접수함 화면.

사건(Incident): 기본 방식은 조사관이 카드를 제출할 때, 인스턴스 방식은 인스턴스를 열 때 업무 진행이 만듦 → 단계(승인 대기 → 명령 → 결과 → 재관측 → 회복 → 작업지시 → 종결, 또는 넘김 · 반려 · 조치 없이 해제)를 옮김 → 업무 진행 SQLite → 대기열을 거쳐 지식 지도 운영 기록 층 → 포털 · 인스턴스의 「회복?」 게이트웨이 · 다음 판단의 선례.

안내 카드: 조사관이 경보 + 지식 지도(원인 · 증거 규칙 · SOP) + 기록 DB(증거) → 언어 모델 요약 → 가드레일 → 업무 진행에 제출(사건의 카드) → 포털 「이상 확인 · 조치」.

판단(조치 카드 묶음): 판단 엔진이 안내 카드 + 입력 데이터(기록 DB · 업무 진행의 설비 상태 · 회사 시스템 · 업무 DB · 지식 지도의 공급사)와 예측 모델로 후보 → 규정 → 순위 → 카드 검사 → 업무 진행에 제출(인스턴스 방식이면 AI 할 일 결과로 Supabase에도) → 포털에서 사람이 봄.

승인 · 검토본: 포털에서 사람이 카드(필요하면 값을 바꾼 검토본, 지문이 붙어 지울 수 없음)를 고름 → 업무 진행이 조사관에게 재평가 요청 → (인스턴스는 승인 전달 기록: 대기 → 전달됨 · 실패 · 폐기) → 통과하면 명령과 작업지시 요청이 정해지고 판단 사례가 지식 지도로. 거절되면 사건은 승인 대기(기본) 작업이 대기(인스턴스)로 감(인스턴스).

명령: 업무 진행이 승인에서 만듦(만료 시각 포함) → Kafka action.cmd → 게이트웨이 검사 → PLC가 아는 이름 · 값으로 바꿔 MQTT plant/설비/cmd/auto(남기지 않음) → 엣지 → PLC 검사 → 설비. 기록 DB 조치 표에도 남음. 게이트웨이 거부는 버리고 감사 기록으로.

ACK: PLC의 처리 결과(완료 · 거절과 이유) → 다음 설비 상태에 실림 → MQTT → 수집 다리 → Kafka plant.status → 업무 진행이 명령 번호로 짝을 맞추(시간 초과 · 거절이면 사람에게) · 저장 심부름꾼이 기록 DB 조치 줄에 첫 결과로 붙임 · FUXA 화면에 표시.

재관측 판정: 업무 진행이 결과를 받은 뒤 기다렸다가 기록 DB의 최신 회복 값 + 경보 해제 여부로 판정 → 회복이면 작업지시로, 아니면 「조치 실패」로 넘김 → 감사 기록 · 사건 단계 → 인스턴스의 「회복?」 게이트웨이가 읽음.

작업지시 · 구매요청(기업 거래 · 원장): 승인 때 정해짐 → 업무 진행(인스턴스면 엔진)이 회사 시스템 목업에 HTTP(같은 요청 열쇠만 한 번만) → memory면 SQLite 원장, supabase면 ent 스키마에 거래와 전후 상태 → CMMS가 돌려준 번호로 사건 종결 → 포털 회사 시스템 보드 · 판단 엔진 입력 · AI 직원(ent 읽기) · 재작업 때 보상 영수증과 대응 거래.

판단 사례 · 선례: 승인 순간(판단 사례)과 사건 단계 변화(사건) → 업무 진행의 SQLite 대기열 → 지식 지도 운영 기록 층(반려는 남지 않음, 나쁜 결과도 남음) → 판단 엔진의 선례 점수 · MCP 서버의 선례 도구.

온톨로지 지식: 스키마 정의 → 지식 채우기 도우미가 Neo4j에 제약 · 인스턴스 · 보강 → 사람의 스킬 · 점수식 편집(영수증) · DDL 등록(입력 데이터와 표 · 칸 연결) · SCM 공급사 주기 동기화 · 매뉴얼 적재(SOP와 단계, 고장 유형에 이르면 후보 규칙에 자동 연결)가 덧붙음 → 검사관(탐지 정의) · 조사관 · 판단 엔진 · MCP 서버 · 포털이 읽음.

프로세스 인스턴스 · 할 일 · 이벤트: 정의서(판본 고정) + 접수한 경보 → 엔진이 Supabase에 인스턴스와 할 일을 만들고 상태를 옮김(예정 → 진행 → 제출됨 → 완료, 또는 대기 · 취소) → AI 직원이나 조사관 다리 · 사람이 할 일을 채움 → 정해진 일은 이벤트 표로(포털 SSE), 일반 상태 이동은 Kafka audit으로 → 지식 지도 실행 층에 비춰 둠.

지식 노드의 출생지: 스키마 정의 → 제약 조건 · OWL 문서 · AI 직원 설명문. 손으로 쓴 인스턴스 파일 → 채우기 도우미 → 가치 · 프로세스 · 리소스 · 진단 · 스킬 · 판단 · 규칙 · 입력 데이터 · 외부 변수 · 예측 · 예시 선례 노드. 매뉴얼 → 절 · SOP 스킬 · 단계 노드(매뉴얼 묶음 소유). DDL → 시스템 · 입력 데이터 노드(DDL 묶음 소유). SCM → 공급사 노드와 견적 관계(업무 DB가 주인). 사람의 편집 → 스킬 · 순위 정책 · 조건 판정 정책 변경과 지식 편집 영수증. 업무 진행 → 사건 · 판단 사례 · 실행 층 노드와 울타리 노드. 소유가 다른 노드는 서로 덮지 않습니다.

판단 규칙 한 줄: 매뉴얼 절 · 설비안전규정 · 구매규정 · BSC 전략맵 → 사람이 쓴 인스턴스(또는 매뉴얼 적재 · 스킬 편집이 후보 규칙에 출력을 이음) → 결정표의 규칙 노드(검사 · 출력 · 적용 대상 · 근거) → 판단 엔진이 사실에 대어 발동 여부를 판정 → 카드의 인용(그 카드를 고른 규칙 · 걸린 규칙과 근거 절) → 포털 카드 · 가정 실험 화면의 규칙별 발동 → 사람이 판단할 때 봄.

입력 데이터의 값: 입력 데이터 노드의 출처 관계(센서 · 기록 DB · SCADA · MES · ERP · CMMS · QMS · SCM · 업무 DB의 표와 칸) → 판단 엔진이 출처대로 읽음(기록 DB SQL · 업무 진행의 설비 상태 · 회사 시스템 HTTP · 공급사 노드 · 업무 DB 직접 SQL) → 값과 「어디서 어떻게 읽었나」 기록 → 규칙 검사와 순위 정책의 재료 → 판단 기록과 카드의 출처 보기.

BSC 득실 경로: 스킬의 AFFECTS → 상태 변수 · 지표 → INFLUENCES 사슬(부호 · 강도 · 조건) → 지표별 득실(조건부는 따로) → 순위 점수의 BSC 성분 → 카드의 득실 경로 → 사람이 보고 고름 → 판단 사례 → 다음 판단의 선례 비율.

검토본: 사람이 카드의 값을 바꿈 → 업무 진행이 조사관에게 새 예측 · 판정을 받음 → 지문이 붙은 검토본을 업무 진행 SQLite에 먼저 저장 → 화면에 보여 줌 → 승인이 그 사본을 그대로 복사(인스턴스 방식이면 승인 전달 행) → 전달 직전 재검사에서 실행 값과 비교 → 명령의 값.

재작업 · 효과 영수증: 사람의 재작업 요청(역할 · 사유 · 확인) → 고정 판본에서 영향 범위 미리보기와 지문 → 이미 일어난 효과마다 보상 거래(회사 시스템 원장에 대응 줄) 또는 확인 영수증 → 업무 기록부의 효과 영수증 · 재작업 영수증(지우지 않음) → 새 세대의 할 일 행 · 되돌린 변수 · 다시 승인 대기인 사건 → 포털 인스턴스 화면 · 실행 중 투영.

회사 원장 줄: 업무 진행의 거래 요청(판단 · 스킬 · 설비 · 값 · 요청 열쇠) → 회사 시스템이 잠금 아래 실행 → 바뀌기 전과 후의 상태 · 참조 번호를 원장에 → 보상 줄은 원래 줄을 가리킴 → 업무 진행의 효과 확인 · 포털 회사 시스템 보드 · AI 직원 (업무 DB 읽기).

AI 직원의 작업 공간: 할 일 행(지시 · 입력) + 폼 · 사용자 · MCP 서버 목록 → 작업 공간 폴더(규칙 문서 · 할 일 · 지시문 · 연결 설정 · 대화 기록) → Claude Code가 MCP 서버로 읽은 근거 → 결과 파일 → 폼 계약 판정 → 할 일 행의 결과 · 이벤트 행(질문 · 보류면 영수증) → 엔진이 다음 작업으로. 폴더는 질문 뒤 이어서 일할 때 다시 쓰이고 보존 기간이 지나면 지워집니다.

실행 중 투영: 업무 기록부의 인스턴스 · 할 일 · 정의 판본 행이 바뀔 → 데이터베이스가 실행 투영 대기열에 한 줄 → 업무 진행이 읽어 판 번호 · 지문과 함께 지식 지도로 → 판본 · 흐름 노드 · 인스턴스 · 작업 노드와 관계(MAPS_TO · USES_VERSION · INSTANCE_OF · ROLE_BOUND · ON_ASSET · HANDLES · IN_INSTANCE · EXECUTES · ASSIGNED_TO) → 포털 인스턴스 화면의 투영 상태 · 지식 지도 질의.

감사 기록: 업무 진행(승인 · 명령 · 재관측 · 상태 이동)과 게이트웨이(통과 · 거부와 검사 이름)가 만듦 → Kafka audit(가장 오래 보관) → 저장 심부름꾼이 기록 DB 감사 표(지우지 않음) → Grafana 감사 패널 · 사람이 사후 확인.

한계 알려진 한계

보안. 게시판(EMQX)에 계정 · 칸별 권한이 없고, 공장과 사무실의 경계도 방화벽이 아니라 프로그램 규칙입니다. 강의용 입구 · 회사 시스템 목업 · 장부 들여다보기 화면에는 인증이 없습니다. 포털의 승인자 이름과 역할은 스스로 적는 값이라 누구든 어떤 역할로든 승인할 수 있습니다. 업무 기록부의 기본 표들은 행 단위 권한을 컷지만 익명 키로 읽을 수 있습니다 (새로 만든 표는 막음). Grafana는 보기 전용으로 열리지만 관리자 로그인으로 고칠 수 있습니다.

데이터 유실. 계측은 QoS 0 · 깨끗한 세션 · 메모리 대기열이라, 수집 다리가 끊긴 동안이나 대기열이 넘친 때, 장부 쓰기가 실패한 때의 값은 남지 않습니다(유실률은 재지 않음). 가벼운 보고에서는 보조 채널이 드물게 와서 초 단위 기록에 빈칸이 생기고, Grafana는 그 빈칸을 선으로 이어 그립니다. 특징값은 경보와 달리 보내기 전에 검사관이 꺼지면 다시 만들지 않습니다. 조사관의 실행 기록, 게이트웨이의 기억, 기본 방식의 타이머는 메모리에만 있습니다. Prometheus와 FUXA는 저장 공간이 없습니다.

어긋남. 시간 배율을 바꿔도 검사관과 업무 진행은 켄 때의 배율로 계산합니다. 캐비닛 온도가 사실상 변하지 않아 「주변 온도 상승」 원인은 증거로 지지되지 않습니다. 운전실 화면의 IT 경보 알람 문구는 「쿨러 성능 저하 (Flink CEP RAISE)」로 고정입니다. 결함 시뮬레이션은 명령 게이트웨이를 거치지 않습니다. 회사 시스템이 기본(memory) 설정이면 AI 직원이

읽는 업무 데이터와 회사 시스템이 쓰는 데이터가 다릅니다.

기동. 지식 채우기 도우미는 검사관의 탐지 정의를 넣지 않습니다. 지식 지도를 빈 저장소에서 새로 채우면 검사관이 읽을 정의가 없어 「준비 안 됨」으로 남고 경보가 나지 않을 수 있으며, 정의를 넣는 별도 작업을 한 번 돌려야 합니다. FUXA는 판을 고정하지 않은 최신 이미지를 받아 새 판이 나오면 동작이 바뀔 수 있습니다. Neo4j MCP 서버는 실행할 때 인터넷에서 내려받습니다. 조사관이 AI 몫을 대신하는 다리(legacy)와 AI 직원을 동시에 켜면 같은 작업을 서로 집으려 다툼니다.

저장소 밖 사실의 출처

이 시스템에 관한 내용은 2026-10-07 코드 · 설정과 날짜가 적힌 실측 기록이 근거입니다. 아래는 이야기에 쓴 일반 사실(표준 · 프로토콜)을 확인한 곳입니다.

- MQTT QoS 0(최대 한 번) · QoS 1(최소 한 번, PUBACK) · retain · clean session — <https://mosquitto.org/man/mqtt-7.html>
- Redpanda: Kafka API 호환 스트리밍 플랫폼 — <https://redpanda.com/>
- Kafka 소비자 오프셋: 그룹마다 읽은 위치를 기록해 재시작 뒤 이어 읽음 — <https://docs.confluent.io/kafka/design/consumer-design.html>
- IEC 61511: 안전 계장 시스템과 일반 제어의 분리 · 독립 — https://www.emersonautomationexperts.com/2009/safety/views_on_safety
- IEC 62443: 구역(zone)과 관문(conduit), IT와 OT 사이 DMZ — <https://wz-it.com/en/knowledge/remote-access/ot-it-segmentation-dmz-remote-access/>
- Modbus: 인증 · 암호화 없는 산업용 프로토콜(현장 구간 설명) — <https://www2.cs.siu.edu/~aydeger/classes/CS%20531/8.ICS-Protocols.pdf>
- EMQX 5.8: MQTT 3.x · 5.0, 대시보드 — <https://docs.emqx.com/en/emqx/v5.8/>
- FUXA 웹 SCADA/HMI — <https://github.com/frangoteam/FUXA>
- Redpanda Console — <https://github.com/redpanda-data/console>
- TimescaleDB 2.17 지원 PostgreSQL 판(14~17) — <https://www.tigerdata.com/docs/self-hosted/latest/upgrades/upgrade-pg>
- Grafana InfluxDB · 익명 보기 — <https://grafana.com/docs/grafana/latest/datasources/influxdb/>
- Prometheus agent 모드(선택 감시 참고) — https://prometheus.io/docs/prometheus/latest/prometheus_agent/
- Neo4j 5.26 LTS · APOC — <https://neo4j.com/blog/developer/neo4j-v5-lts-evolution/>
- Neo4j MCP 서버(mcp-neo4j-cypher) — <https://github.com/neo4j-contrib/mcp-neo4j>
- MCP 전송 방식: stdio · Streamable HTTP — <https://modelcontextprotocol.io/specification/2025-06-18/basic/transport>
- Claude Code CLI: 비대화 실행 · 허용 도구 · .mcp.json — <https://code.claude.com/docs/en/cli-reference>
- Supabase 로컬 개발(API · PostgreSQL · Studio) — <https://supabase.com/docs/guides/local-development/cli/config>
- SQLGlot: SQL 구문 분석기 — <https://github.com/tobymao/sqlglot>
- aiokafka(asyncio Kafka 클라이언트) — <https://github.com/aio-libs/aiokafka>
- BSC 네 관점(재무 · 고객 · 내부 프로세스 · 학습과 성장)과 전략맵의 인과 — <https://bscdesigner.com/four-perspectives.htm>
- DMN 결정표 적중 방식(Unique · Priority · First · Collect 등) — <https://www.trisotech.com/dmn-hit-policy-explained/>
- BPMN 2.0 경계 타이머 이벤트 · 배타 게이트웨이 · 사용자/서비스 작업 — <https://www.ibm.com/docs/SS964W/com.ibm.wbpm.admin.doc/topics/mappingBPMNconstructs2WBPM.html>

- ISO 13374 / MIMOSA OSA-CBM: 상태 감지 · 건강 평가 · 예측 · 권고 블록 — https://www.mimosa.org/wp-content/uploads/2006-08-28_Including_OSA-CBM_and_OSA-EAI_in_Integrated_System_Health_Management.pdf
- ISO 14224: 설비 분류 · 고장 모드와 원인 · 정비 데이터 — <https://www.fabrico.io/blog/iso-14224-failure-taxonomy-codes/>
- SQLite WAL — <https://www.sqlite.org/wal.html>